Munich University of Applied Sciences
Faculty of Computer Science and Mathematics



Master Thesis
for obtaining the academic degree of
Master of Science
in course of studies Computer Science

Observing an Identity Provider from Within: Design, Implementation and Evaluation of a Security Monitoring System for Shibboleth

---

Author:	Tadashi Toyokawa
Matriculation number:	27740221
Submission Date:	4. November 2026
Supervisor:	Prof. Dr.-Ing. Thomas Schreck
Second Supervisor:	Prof. Dr.-Ing. Erik Krempel
Advisor:	Erwin Kupris

I confirm that this master thesis is my own work and I have documented all sources and material used.

Munich, 4. November 2026 Tadashi Toyokawa

Acknowledgments

I would like to express my deepest gratitude to my advisor, Erwin Kupris, my supervisor Prof. Dr.-Ing. Thomas Schreckand my second supervisor Prof. Dr.-Ing. Erik Krempel, for their invaluable guidance, insightful feedback, and continuous support throughout this research. Furthermore, I extend my appreciation to my family and friends for their encouragement and support during this journey. Their motivation and belief in my abilities have been a constant source of strength.

Abstract

In a federated identity infrastructure, a single identity provider authenticates users on behalf of many service providers and thereby becomes the trust anchor of the entire federation. A compromised or misconfigured provider does not affect one application but can undermine the security of every service relying on it. Despite this exposure, systematic security monitoring at the application layer of such providers remains largely absent, and existing approaches in this domain focus on operational availability rather than on security.

This thesis presents a monitoring system that observes a Shibboleth identity provider from within its own application layer. The system draws on three sources that differ in what they can observe, namely the audit log written at the end of a transaction, the configuration and metadata declared for each service provider, and the runtime state accessible through interceptors executing inside the authentication flow. Records from these sources are pseudonymised at their origin, transmitted to a backend located outside the trust boundary of the provider, correlated with one another and evaluated against a catalogue of fifteen detection rules. The rules are derived from vulnerabilities and attack classes documented in a systematic survey of federated identity security and from the security recommendations of the OASIS SAML Security and Privacy Considerations.

The evaluation examines how reliably the records can be correlated, whether the detection rules behave as specified, and what the resulting system costs. The correlation reaches a coverage of 100 % over 7,236 reports. Where the audit log records the identifier of the request, it assigns every report unambiguously, independent of the login density. Where the identifier is missing, the heuristic strategies remained free of incorrect assignments in every tested scenario as long as the subject or the address separates the users, and the strategy based on time alone confused only logins at the same service provider less than 100 ms apart. All fifteen rules were verified against a real search index and six additionally through the complete pipeline from a browser login, four of them up to the resulting alert. In the asynchronous transmission mode, the end-to-end login latency did not differ measurably from a deployment without interceptors, with an upper bound of 17 ms for the 95% confidence interval, below five percent of the median login duration. Instead, the interceptors cost about 28 ms of processor time per login, which the IdP spends in background threads, whereas blocking transmission added about 60 ms to each login without saving processor time.

The work also identifies the boundaries of the chosen observation point. Attack classes located at a different system layer, involving a different party, or observable only through their preconditions remain outside its reach, and a provider abusing its own position cannot be detected by a system executing inside it. Beyond these results, each of the three evaluations uncovered a defect in the infrastructure or the tooling of this work that operation itself had not revealed, which illustrates the premise underlying the work, namely that a system appearing to function is not thereby known to function.

Contents

1 Introduction **1**

2 Background **3**
2.1 Technical Foundation of Federated Identity Management 3
2.2 SAML 4
2.2.1 SAML Assertions 4
2.2.2 XML-Signature and Encryption 5
2.2.3 SAML Bindings 6
2.2.4 Web Browser SSO Authentication Process 6
2.2.5 SAML Metadata 8
2.3 Shibboleth 8
2.3.1 Profile Flows and Interception Points 9
2.3.2 The Context Tree 9
2.3.3 Audit Logging 10
2.3.4 Configuration 10

3 Related Work **11**
3.1 Security Analyses of SAML and Shibboleth 11
3.2 Why Monitor from the Inside? 11
3.3 Why SIEM Functionality Is Not Enough 13
3.4 Derivation of the Rules 15

4 System Design **17**
4.1 Requirements for the System 17
4.2 Overview 18
4.3 Data Aggregation Layer 19
4.3.1 The Audit Logs 19
4.3.2 Metadata and Relying Party File 20
4.3.3 Interceptor 21
4.4 Transport Layer and Trust Boundary 23
4.5 Correlation and Detection Layer 25
4.5.1 Correlation 25
4.5.2 Detection 26
4.5.3 Severity Classification 28
4.6 Visualisation Layer 29
4.7 Design Decisions and Rejected Alternatives 30

5 Implementation **33**
5.1 Data Aggregation Layer 33
5.2 Transport Layer 35

5.3 Correlation and Detection Layer 36
5.3.1 Storage 36
5.3.2 Correlation 37
5.3.3 Detection 39
5.4 Visualisation Layer 40

6 Evaluation **43**
6.1 Test Setup 43
6.2 Correlation 43
6.2.1 Method and Datasets 44
6.2.2 Results 45
6.3 Detection 48
6.3.1 Method and Scenarios 49
6.3.2 Results 50
6.4 Performance Overhead 53
6.4.1 Method and Measurements 53
6.4.2 Results 54
6.5 Defects Found 56
6.6 Threats to Validity 56

7 Discussion **59**
7.1 Interpretation of the Results 59
7.2 Limitations of the Approach 60
7.3 Implications for Future Research 62

8 Conclusion **65**

Glossary **69**

1 Introduction

In today's increasingly digital world, where most online services require users to create an account to access them, users are forced to create and memorise a large number of different credentials, or are even forcibly encouraged to reuse old passwords on multiple services. These online profiles are highly sensitive, as they are typically associated with the user's personal identity, including details such as their email address, phone number or even official government documentation like a personal ID. More often than not, this profile is protected by a single password created by the user. This dominant method of password-based authentication, which is responsible for verifying and authenticating users online, poses a significant risk to companies and other large organisations. One way in which malicious attackers take advantage of this vulnerable authentication method is by using large databases of leaked user credentials (often including email addresses and corresponding passwords) to authenticate at Service Providers (SPs) as the affected user. This attack method is called credential stuffing. Verizon reports that credential abuse was the leading initial attack vector in 22% of the 12,195 confirmed data breaches in the 2025 report [**VerizonDataBreach**].

To reduce the number of places where credentials are created and stored, Federated Identity Management (FIM) was developed. FIM lets users access several SPs with a single set of credentials that is managed by an Identity Provider (IdP). FIM consolidates authentication behind a single trusted authority called the IdP. When a user attempts to gain access on the SP in FIM, the user's browser is redirected to the IdP. The user authenticates with the IdP, which then confirms the user's identity to the SP [**FIMSurvey**].

Shibboleth, developed by Internet2, is one of the most widely deployed open-source implementations of this model in the research and education sector [**Shibboleth**]. Under this model, a user authenticates with their organisation's IdP once. The IdP then issues cryptographically signed SAML assertions that SPs accept as proof of identity. This architecture is widely deployed in higher education and research federation such as DFN-AAI [**DFNAAI**], where a single IdP serves dozens of SPs simultaneously.

This architectural property introduces a critical consequence: the IdP becomes a single point of trust for the entire federation [**IdPSinglePointOfTrust**]. A compromised, misconfigured or vulnerable IdP does not affect one application. It can undermine the security of every relying SP. A notable example is the Sunburst supply chain attack [**FireEyeSSC**]. As Hartl and Derek note, after the initial compromise the attackers obtained the trusted certificate used for signing SAML assertions and used it to access all services within the targeted enterprise [**FormalSecAnalysisSAML**]. Since tokens forged with this certificate carry the organisation's own trusted signature, the resulting anomalies may go unnoticed [**MicrosoftBlogSSC**]. Beyond targeted attacks, SAML deployments are exposed to weaknesses in the protocol and in its implementations. Somorovsky et al. found XML Signature Wrapping (XSW) vulnerabilities in eleven of fourteen SAML frameworks, including Shibboleth, which allow an attacker to inject forged claims that the SP accepts once the signature has been validated [**BreakingSAML**], and the attack has since been

reproduced in other OpenSAML-based setups [**VulnerSAMLCloud**]. In a formal analysis of Shibboleth, Sommer identified three further attacks, namely Response Injection, Leaked Assertion Artifact and IdP Spoofing, which let an attacker log a victim into an account controlled by the attacker or reuse a leaked artifact to log in as the victim. The Shibboleth developers attribute the first of these to the SAML standard itself rather than to the implementation [**SecAnalysisSSO**].

Despite this elevated risk profile, systematic security monitoring at the application layer of Shibboleth IdP deployments remains largely absent. Existing approaches such as AAIEye [**MonitoringAAI**] focus primarily on operational availability, leaving security-relevant aspects unaddressed. The research literature reflects a comparable gap. A survey of security analyses in federated identity management finds Shibboleth to be among the least examined protocols and raises the question of whether this reflects a stronger security posture or merely a lower level of attention [**FIMSurvey**]. Formal analyses of SAML likewise remain scarce despite its deployment across government, enterprise and education [**FormalSecAnalysisSAML**].

This thesis addresses this gap by proposing a security-oriented monitoring system that operates directly within the Shibboleth application layer. It makes application-layer security parameters and authentication workflows visible that are otherwise inaccessible from outside the application. Rather than testing the deployment actively, the system observes it passively. This principle has been applied to the eduroam infrastructure by analysing its network traffic [**EAPeduroam**], whereas this work applies it inside the application, at the application layer of the IdP. The system thereby provides federation operators with a continuous view of the security posture of their authentication infrastructure.

Three research questions guide this work:

- RQ1 Which security-relevant data can be collected at the application layer of a Shibboleth IdP, and through which mechanisms are these data accessible?
- RQ2 How can these data be correlated in order to detect configuration weaknesses and anomalies in the authentication process?
- RQ3 What measurable overhead does the resulting system impose on the monitored IdP?

The remainder of this thesis is structured as follows.

- Chapter 2 introduces the technical foundations of FIM, SAML and the Shibboleth IdP.
- Chapter 3 positions this work within existing research on application layer intrusion detection and delimits it from network and host based monitoring systems.
- Chapter 4 presents the architecture of the proposed system.
- Chapter 5 describes its implementation, divided by layer.
- Chapter 6 evaluates the correlation, the detection rules and the overhead of the system.
- Chapter 7 interprets the results with respect to the research questions and discusses the limitations of the approach.
- Chapter 8 summarises the findings.

2 Background

This chapter explains the key concepts and terminology relevant to this thesis and establishes the technical foundation required for the design of the proposed monitoring tool. It is limited to the aspects on which the later chapters build, and consequently covers neither the full range of the SAML specification nor the complete configuration surface of the Shibboleth IdP.

2.1 Technical Foundation of Federated Identity Management

As introduced in Chapter 1, FIM allows users to access the resources they need with a single digital identity and provides Single Sign On (SSO) functionality across the applications of an organisation. The immediate benefit is convenience, since users no longer need to create and remember a separate set of credentials for every service. The more consequential benefit lies elsewhere. Because credentials exist in one place rather than in every participating service, the number of systems that store and verify them is reduced, since the RP neither verifies the user's authenticators itself nor needs to issue its own [**NIST80063C**]. The strength of authentication is consequently also determined in one place. An RP can require a minimum authentication assurance level, which the IdP fulfils and reports in the assertion [**NIST80063C**], and from the second level upwards this requires two distinct authentication factors [**NIST80063B**]. Concentrating authentication at one well-managed site also focuses administrative effort and makes multi factor authentication more acceptable to users, who encounter it less often [**ShibbolethGIAC**]. Administration follows the same pattern, since accounts are created, modified and revoked centrally rather than in each service individually.

This centralisation introduces a new risk in return. FIM rests on a relationship of trust between organisations, and that trust depends entirely on the security of the IdP. Concentrating credentials in one place also concentrates the consequences of a compromise, since an attacker who gains control of the IdP can potentially authenticate as any of its users at the services that rely on it [**IdPSinglePointOfTrust**].

A compromise of a primary IdP could potentially grant broad unauthorised access across the entire federation. This characteristic requires both the IdP and SP to maintain a high level of security [**NIST80063C**]. The policies defined for each SP in the IdP must accurately translate to the rules at the corresponding SP. The configuration of the IdP and that of each SP must remain consistent with one another. Certificates, keys and protocol settings are maintained separately on both sides, and a mismatch between them can weaken the security of a transaction without either party noticing. To better understand the interaction within a federated identity system, the roles and responsibilities of the IdP and SP are described in the following.

A federation consists of two kinds of participants. The IdP authenticates users and issues statements about them [**SAMLOverview**]. The SP offers the resource a user wants

to reach and relies on those statements instead of authenticating the user itself. Neither role is meaningful without the other, and their interaction follows a fixed pattern.

When a user requests a protected resource, the SP does not check credentials. It determines which IdP is responsible for the user and redirects the browser there together with an authentication request [**SAMLOverview**]. The IdP authenticates the user by whatever means the deployment provides, and returns a signed statement asserting that the authentication took place and describing the user by a set of attributes. The SP verifies the signature, evaluates the statement and grants or denies access on that basis. The credentials themselves never leave the IdP.

This division has a consequence for how a federation is configured. Because the IdP serves many SPs, it does not treat them uniformly. It maintains a separate configuration for each partner, specifying for instance whether the statement is to be encrypted, which authentication methods are permitted, and which attributes are released. The SP maintains its own configuration in turn, stating what it expects to receive and what it will accept.

Both sides therefore hold configuration that describes the same relationship, and both are maintained independently. What binds them together is the metadata each party publishes about itself. This includes the endpoints at which it can be reached, and the certificates by which its statements can be verified. Trust within a federation rests on these metadata being current and correctly exchanged [**SAMLMetadata**]. Whether the two configurations remain consistent with one another, however, is not something the protocol itself verifies.

2.2 SAML

The Security Assertion Markup Language (SAML) is an XML-based standard developed by the Organization for the Advancement of Structured Information Standards (OASIS) for the exchange of authentication, authorisation, and other security-related information between different parties. SAML is a fundamental component of FIM and is widely used to enable SSO across organizational and security boundaries [**SAMLOverview**]. The second major version, SAML 2.0, was published by OASIS in March 2005 [**SAML2Standard**]. SAML 1.1 had already gained widespread adoption in federated identity environments [**SAML1Standard**]. However, several competing approaches existed at the time, including the Identity Federation Framework (ID-FF) developed by the Liberty Alliance and the Shibboleth architecture [**SAML2Standard**]. SAML 2.0 consolidated several of these approaches into a common standard and introduced additional capabilities, particularly with regard to federation, interoperability, and privacy.

2.2.1 SAML Assertions

SAML assertions serve as security tokens that convey security-related information about a principal, such as a user, system, or service. An assertion contains statements made by an issuer about the principal, which may include authentication, authorisation, and attribute information. Since these statements can have security-critical implications, their

authenticity and integrity must be ensured [**SAMLAssertions**]. Therefore, the SAML element can be protected by a digital signature following the XML Signature standard, allowing a relying party to verify that the assertion was issued by the stated issuer and has not been modified [**BreakingSAML**].

Each assertion contains metadata that defines its origin and validity. The `Issuer` element identifies the entity that issued the assertion, while the `IssueInstant` attribute specifies the time at which it was issued. Furthermore, every assertion is assigned a unique identifier that must not be reused. Once an assertion has been issued, the issuer has no direct control over its subsequent use. Consequently, a consumer of an assertion is expected to enforce its specified validity conditions, including any validity interval and applicable `OneTimeUse` condition [**SAMLAssertions**].

2.2.2 XML-Signature and Encryption

To protect the integrity and authenticity of SAML assertions, XML signatures can be used to cryptographically bind the assertion to its issuer. The W3C XML Signature standard defines how signatures over XML data are created, encoded and verified [**W3CXMLSIG**]. A signature is defined by the `Signature` element and can be applied to an entire SAML assertion or to selected parts of an XML document [**SAMLAssertions**, **BreakingSAML**].

Depending on the location of the signature relative to the signed data, XML signatures can be categorised into three types [**W3CXMLSIG**]. An enveloped signature is contained within the XML element that it signs. In contrast, an enveloping signature contains the signed data within the signature itself. A detached signature is kept apart from the signed data, which it neither contains nor is contained in, and refers to that data by reference [**W3CXMLSIG**]. In the context of SAML, enveloped signatures are commonly used to sign SAML assertions and protocol messages [**SAMLAssertions**].

XML Encryption provides confidentiality for sensitive information contained in SAML messages. Instead of transmitting the original XML content in plaintext, the protected content is replaced by an `EncryptedData` element containing the resulting ciphertext [**W3CXMLENC**]. The encryption therefore enables selective and persistent confidentiality of individual elements within an XML document. This allows, for example, sensitive attributes contained in a SAML assertion to be encrypted without requiring the entire message to be encrypted.

Although XML Signature and XML Encryption provide important security mechanisms, their use and configuration depend on the respective SAML profile, deployment, and implementation. Therefore, SAML is not a security protocol in itself, but rather a framework whose optional features can be combined in various ways. The resulting security properties depend on the chosen combination. Hartl and Derek make this explicit by deriving eight protocol variants from the optional features of a single use case and analysing each separately [**FormalSecAnalysisSAML**]. Consequently, the security properties of a SAML-based system cannot be derived solely from these standard mechanisms. Incorrect configuration or implementation may result in insufficient protection, such as assertions that are not appropriately signed or sensitive information that is transmitted without encryption.

2.2.3 SAML Bindings

SAML bindings define how SAML protocol messages are transported using underlying communication protocols. They are particularly relevant for the Web Browser SSO profile, where the user's web browser acts as an intermediary between the IdP and SP. The SAML specification defines several bindings for different communication scenarios. In a typical SP-initiated Web Browser SSO process, the HTTP-Redirect and HTTP-POST bindings are particularly relevant [**SAMLBinding**].

2.2.3.1 HTTP-Redirect Binding

The HTTP-Redirect binding is designed for scenarios in which SAML requester and responder communicate through an HTTP user agent acting as an intermediary. In the SP-initiated SSO flow, the SP acts as the SAML requester and the IdP as the SAML responder. The authentication request is encoded and transmitted as part of the URL query string of an HTTP redirect. The user's browser follows the redirect and thereby forwards the request to the IdP.

Since the message is transported through the URL using the HTTP GET method, the binding is particularly suitable for relatively small SAML messages, such as an authentication request. The HTTP-Redirect binding therefore provides the mechanism by which the SP can initiate the authentication process without requiring a direct communication channel between the SP and IdP [**SAMLBinding**].

2.2.3.2 HTTP-POST Binding

The HTTP-POST binding is likewise designed for communication between a SAML requester and responder through an HTTP user agent. In the return direction of the typical SP-initiated SSO flow, the IdP transmits the SAML response to the SP. The SAML response is embedded in an HTML form and sent to the SP using the HTTP POST method. The user's browser acts as an intermediary by submitting the form to the SP.

Compared with the HTTP-Redirect binding, HTTP-POST is better suited for larger SAML messages because the message is not encoded into the URL. This makes it particularly suitable for SAML responses containing assertions and associated security-related information [**SAMLBinding**].

2.2.4 Web Browser SSO Authentication Process

The SAML Web Browser SSO profile enables a user to authenticate to an IdP and subsequently access a protected service provided by an SP. The process is mediated by the user's web browser, which forwards SAML messages between the SP and IdP.

The process begins when a user requests access to a protected resource at the SP. If the user does not have an established session, the SP generates a SAML authentication request and redirects the user's browser to the IdP. The IdP processes the request and authenticates the user using the configured authentication mechanism. Upon successful

authentication, the IdP generates a SAML response containing a SAML assertion with security-related statements about the authenticated principal.

The assertion can be digitally signed by the IdP to provide authenticity and integrity. The SAML response is then returned to the user's browser, which forwards it to the SP. The SP validates the received SAML response and assertion, including the digital signature and the applicable validity conditions. If the assertion is considered valid, the SP evaluates the contained statements and attributes to determine whether the user is authorised to access the requested resource. If the authorisation requirements are satisfied, the SP grants the user access to the protected service [**SAMLOverview**]. Figure 2.1 illustrates the described authentication process.

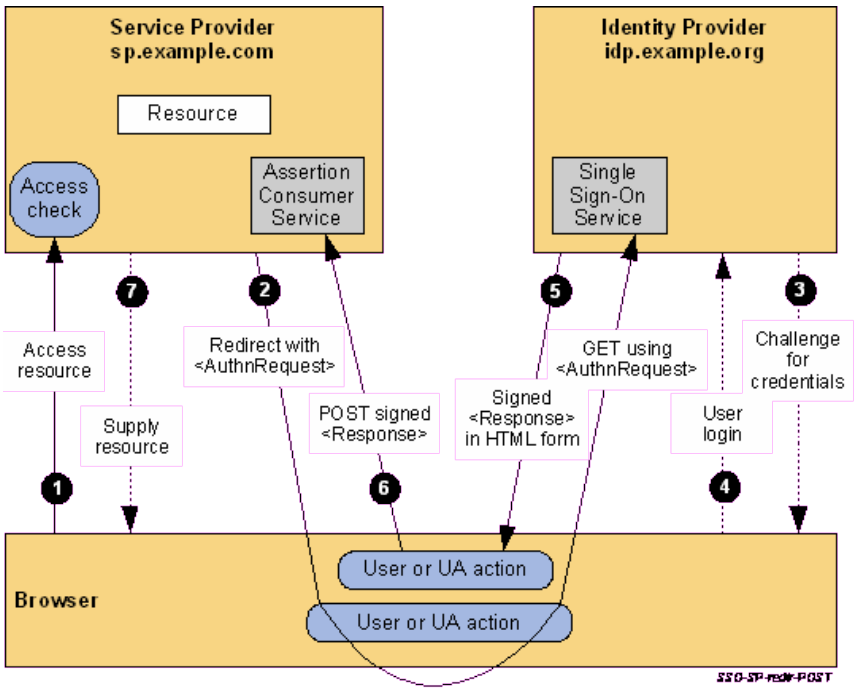


Figure 2.1: *SP-Initiated SSO with Redirect and POST Bindings [***SAMLOverview***]*

The use of the user's web browser as an intermediary introduces an important security consideration. The SAML messages exchanged between the IdP and SP are transmitted through the user's browser and therefore pass through a communication path that is not directly controlled by either party. Neither the IdP nor the SP can assume that the messages remain unchanged while being forwarded. Consequently, the security of the protocol cannot rely solely on the communication channel. Instead, mechanisms such as digital signatures, message identifiers, validity conditions, and audience restrictions are used to ensure the authenticity and integrity of SAML messages and to prevent their misuse [**SAMLSecurityConsideration**].

An important security property of this process is the prevention of message replay. SAML requests and assertions contain unique identifiers that allow the SP and IdP to assign messages and detect previously processed messages. In particular, an assertion's

unique identifier must not be reused, and the recipient is expected to enforce the applicable validity conditions and replay-related constraints.

2.2.5 SAML Metadata

SAML metadata provides a standardised way for SAML entities to describe their configuration and capabilities to other participating entities. It enables IdPs and SPs to obtain information required for establishing communication and processing SAML messages without requiring every configuration parameter to be exchanged manually. The SAML metadata specification organizes this information around roles supported by an entity, which are represented using role descriptors within an `EntityDescriptor` element [**SAMLMetadata**].

For an IdP, the `IDPSSODescriptor` describes information relevant to its role as a SAML IdP. Similarly, an SP can be described using an `SPSSODescriptor`. These descriptors contain information such as supported SAML protocols, communication endpoints, supported bindings, and cryptographic keys. For example, an SP metadata document can specify its Assertion Consumer Service (ACS) endpoint and the bindings supported for receiving SAML responses, while an IdP can specify its SSO Service endpoints and the corresponding supported bindings.

Cryptographic information contained in metadata is particularly important for the security of SAML communication. Metadata can contain certificates and public keys used by participating entities for verifying digital signatures or encrypting SAML messages. Consequently, the metadata configuration directly influences how SAML messages are validated and which endpoints and cryptographic keys are considered valid by an implementation.

The security of the metadata itself is therefore an important aspect of a SAML deployment. If an entity accepts manipulated metadata, an attacker could potentially influence endpoints, supported bindings, or cryptographic keys used during SAML message processing. SAML metadata can itself be protected using digital signatures, allowing an entity to verify the integrity and origin of the metadata before using it for configuration or trust decisions [**SAMLMetadata**].

2.3 Shibboleth

Shibboleth is a free and open-source web single sign-on software suite that implements SAML and other standards for FIM. It is widely used in federated authentication environments and places particular emphasis on privacy and decentralised identity management. The Shibboleth Consortium provides separate software components for the IdP and SP, enabling organizations to participate in federated identity infrastructures [**Shibboleth**].

For the scope of this thesis, the Shibboleth IdP is of particular relevance, as it is responsible for authenticating users and issuing SAML assertions to participating SPs. The IdP is implemented as a Java-based web application and is typically deployed using a Java servlet container such as Apache Tomcat. User identities and associated attributes are commonly managed through an external directory service, such as a Lightweight Directory

Access Protocol (LDAP) server, which provides the IdP with the information required for authentication and attribute release. It builds on the OpenSAML library for the construction and validation of SAML messages, which is relevant because vulnerabilities reported for OpenSAML can affect Shibboleth deployments as well [**ShibbolethGIAC**].

2.3.1 Profile Flows and Interception Points

The IdP processes each request as a workflow built on Spring Web Flow, a framework in which a sequence of processing steps is described declaratively rather than in program code. A flow consists of states that are traversed in order, and a state may delegate to a subflow, that is a self-contained flow which performs a defined task and returns control to its caller. Every top-level flow of the IdP implements one of the supported protocol interfaces, referred to as profiles, such as SAML 2 SP-initiated SSO. Recurring tasks within such a profile, among them the authentication of the user, are carried out by subflows [**ProfileHandling**].

To allow developers to extend this behaviour without copying or editing the internal flow definitions, the IdP provides a set of defined hooks. A subflow attached to such a hook is called an interceptor, and three hooks exist:

1. inbound message processing
2. post-authentication processing and
3. outbound message processing

The first and third are available in any profile flow, whereas the second applies only to profiles that actually authenticate a user [**ProfileHandling**]. An interceptor is registered as a bean and enabled by adding its flow identifier to the corresponding list property of a profile configuration in `conf/relying-party.xml`. No modification of the IdP source code is required [**ProfileInterceptConfiguration**].

The choice of hook determines what an interceptor can observe. An inbound interceptor sees the incoming message context but not the outcome of authentication, since that has not yet taken place. An outbound interceptor sees the state of the tree produced by the profile execution, including the outgoing message. The post-authentication hook runs in the middle of processing, after the user has been authenticated but before the outgoing message is constructed. It can therefore observe both the incoming request and the authentication result, but not the response that is issued afterwards [**ProfileInterceptConfiguration**].

2.3.2 The Context Tree

While a flow describes the sequence of processing steps, the state of the request being processed is held separately in a structure referred to as the context tree. Its root is the `ProfileRequestContext`, which is created when a request arrives and accompanies it through the entire profile execution. Attached to this root are child contexts, each holding one aspect of the request, so that the state grows as processing advances rather than being assembled in a single object at the outset.

Three of these child contexts are relevant in the following. The `InboundMessageContext` holds the message as it was received, the `RelyingPartyContext` identifies the SP the request originates from and records whether that identity could be verified, and the `SubjectContext` holds the authenticated subject once authentication has completed [**ProfileHandling**]. The last of these illustrates the incremental nature of the structure, since it does not exist before the user has been authenticated.

2.3.3 Audit Logging

The IdP writes one line per completed transaction to `idp-audit.log`. The set of fields is not fixed but defined by the operator in `conf/audit.xml`, so the information available in the log depends on the configuration of the individual deployment. In the default configuration, authentication attempts that do not result in a completed transaction, in particular failed password validations, produce no entry in this log. The IdP can additionally record every authentication attempt, but this feature is disabled by default, and when enabled its records enter the same audit stream unless they are routed to a separate destination [**AuditLoggingConfiguration**].

The format of a log line is defined per profile in the *shibboleth.AuditFormattingMap* of `conf/audit.xml`. A format consists of formatting tokens, each of which is replaced by a value from the context tree of the transaction when the line is written, for instance the time of the event, the identifier of the inbound message or the address of the client. Some values are written in an abbreviated form, which the IdP derives from a replacement map (*shibboleth.AuditFieldReplacementMap*), and further tokens can be added by the operator [**AuditLoggingConfiguration**].

2.3.4 Configuration

Three configuration files are relevant in the following. `relying-party.xml` holds the per-partner configuration described in Section 2.1, including which profiles are enabled and which security settings apply [**RelyingPartyConfiguration**]. `metadata-providers.xml` declares where the IdP obtains metadata for its partners [**MetadataConfiguration**]. `attribute-filter.xml` governs which attributes are released to which relying party [**AttributeFilterConfiguration**].

The settings of `relying-party.xml` apply to all partners by default and can be overridden for a single partner, so that one partner is configured differently from the others. They include the authentication flows permitted for a partner and whether responses and assertions are signed or encrypted [**RelyingPartyConfiguration**].

3 Related Work

Existing research and implementations were examined to identify approaches relevant to the development of a monitoring system for an IdP operating at the application layer. The primary objective was to identify solutions that address the overarching goal of this thesis and to analyse their respective limitations and areas of applicability.

These related works provide valuable insights and a foundation for this thesis by identifying existing approaches, their areas of coverage, and gaps that remain relevant to the considered scenario. The most relevant studies and projects are presented and analysed in the following sections.

3.1 Security Analyses of SAML and Shibboleth

The security of SAML-based federations has been examined from two directions. Empirical studies analyse existing implementations, while formal approaches reason about the specification itself.

Somorovsky et al. examined fourteen SAML frameworks and found critical XSW vulnerabilities in eleven of them [**BreakingSAML**]. Their analysis is confined to the processing of assertions at the SP, and it does not consider what information is available at the IdP during or after such an attack. Kaur and Bansal transferred this line of work to a cloud setting by reproducing an XSW attack in their own OpenSAML-based SSO environment and rating its severity with CVSS, which fell from 6.2 to 2.7 once a mitigation was applied [**VulnerSAMLCloud**].

A second line of work approaches the problem formally. Sommer analyses the Shibboleth implementation with respect to authentication, session integrity and logout consistency [**SecAnalysisSSO**], while Hartl and Derek analyse the SAML specification itself [**FormalSecAnalysisSAML**]. The latter derive eight protocol variants from the optional features of a single use case and prove the security properties of each separately. Their premise is that SAML is not a security protocol in itself, but a framework whose optional features can be composed in different ways.

Both approaches establish which configurations are secure under a given set of assumptions. Neither verifies whether a deployed system actually satisfies those assumptions. Formal analysis therefore answers which configuration is secure, whereas this work observes which configuration is in effect.

3.2 Why Monitor from the Inside?

As discussed in the Chapter 1, the proposed monitoring system operates directly within the application layer of the Shibboleth IdP. Its objective is to detect attacks based on events and behaviour observable from within the application.

The underlying concept is not new. The OWASP AppSensor project describes an approach in which intrusion detection and response capabilities are integrated directly into

an application [**OWASPAppSensor**]. In contrast to network-based security mechanisms, application-layer monitoring can make use of information about the application's internal state and semantics.

A network firewall primarily controls network communication, while a network-based Intrusion Detection System (IDS) has limited visibility into application-specific behaviour. A Web Application Firewall (WAF) can provide additional protection against generic application-layer attacks, such as SQL injection, but typically has limited knowledge of the internal logic and state of a specific application. Consequently, application-integrated detection mechanisms can complement network- and perimeter-based security controls by analysing application-specific events and behaviour [**OWASPAppSensor**].

This application-level perspective also provides a basis for classifying the proposed system within an established security taxonomy. In the following, the proposed system is classified using the taxonomy for self-protecting software systems introduced by Yuan and Malek [**Taxonomy**]. Two dimensions of this taxonomy are particularly relevant to the scope of this work: the `Architecture Level` and the `Self-Protection Level`.

The `Architecture Level` describes the layer at which a security mechanism operates and distinguishes between Network, Host, Software/Application, and Services. This dimension allows the observation position of different security mechanisms to be compared. Network-based IDS, such as Suricata and Snort, operate primarily at the network level, whereas Endpoint Detection and Response (EDR) solutions operate at the host level. The proposed system operates at the Software / Application level, as it observes events directly within the Shibboleth IdP. The distinction between these systems therefore lies primarily in the layer at which security-relevant information becomes observable.

While the Architecture Level describes where the system operates, the `Self-Protection Level` describes the degree of self-protection according to the MAPE process of Monitor, Analyze, Plan, and Execute. The proposed system fully covers the *Monitor & Detect* level by collecting and detecting security-relevant events within the IdP. The *Analyze & Characterise* level is partially addressed through event correlation and severity classification. In contrast, *Plan & Prevent* is outside the scope of this work. The system generates alerts based on detected events but does not autonomously modify the system state or take preventive countermeasures.

This limited level of self-protection is intentional and follows from the passive design principle of this work. The proposed system follows the principle of passive observation, in which security-relevant events are made observable without interfering with the operation of the monitored system. Rieckers applies the principle of passive observation to network traffic, capturing the RADIUS and EAP-TLS traffic of the eduroam infrastructure to analyse the TLS parameters that clients offer [**EAPeduroam**]. In captured traffic, information that exists only inside the application, such as the structure of a SAML assertion or the configuration of an SP, is not visible. This work adopts the principle and applies it inside the Shibboleth IdP itself. Extending the system with automatic response mechanisms would contradict this passive observation principle. Furthermore, erroneous automated responses could negatively affect the availability or operation of the IdP and thereby potentially impact multiple SPs that rely on it.

The principle of passive observation is also reflected in existing application-level monitoring approaches. One relevant architectural precedent is BlackWatch by Hall et al. [**BlackWatch**]. BlackWatch applies a similar architecture to generic web applications. Detection points are placed directly within the application to identify security-relevant events. These events are collected by the separate BlackWatch application, where they are analysed using rule-based and reputation-based mechanisms. The results are then made available through a web-based reporting interface.

The separation between the monitored application and the analysis component allows the detection and analysis functionality to be decoupled from the application itself. A key challenge identified by Hall et al. is the integration and configuration of detection points within the application. Developers must identify suitable locations where suspicious behaviour can be detected and implement the corresponding detection logic [**BlackWatch**].

A domain-specific precedent is provided by Suvanto's AAIEye, a monitoring system for Shibboleth- and SAML 2.0-based federations [**MonitoringAAI**]. Motivated by the lack of effective monitoring solutions for authentication and authorisation infrastructures, AAIEye deploys probes alongside the monitored IdP and SP components. These probes collect two categories of information that closely resemble the sources used in this work: runtime authentication events and federation metadata.

However, the evaluation applied to this data differs fundamentally. The usage statistics probe records an authentication event only as a data point for a login count, without assessing whether the event itself is suspicious. The metadata probe verifies only the currency of a metadata file, through a checksum comparison against a reference copy, not its security-relevant content such as certificate validity or endpoint configuration. Suvanto notes that problems in federation operation are often caused by configuration mistakes or changes that cannot currently be easily detected [**MonitoringAAI**]. This is precisely the gap addressed in this work. The same categories of data that AAIEye already collects for operational purposes are evaluated here for their security semantics instead.

A complementary perspective is provided by Martín et al., who propose behavioural anomaly detection at the SP rather than the IdP [**UEBAFederation**]. Their argument is that delegating authentication to an IdP does not transfer the entire security responsibility, and that the relying party can contribute to detection even after authentication has succeeded. The IdP side is explicitly outside their scope, and protocol-level information such as assertion structure, attribute release policies or SP metadata is consequently not considered. Taken together with the present work, the two perspectives address opposite ends of the same federation.

3.3 Why SIEM Functionality Is Not Enough

A Security Information and Event Management (SIEM) primarily operates on logs and security-relevant events. This is not a criticism, but a description of its basic operating principle. Two limitations follow from this principle. First, information that is not represented in an event or log is not available to the SIEM for analysis. Second, the SIEM

generally relies on the information provided by its data sources and therefore cannot independently verify information that is incorrect or has been manipulated at the source.

These limitations do not diminish the value of SIEM systems, but they define the boundary within which they operate. The collection, storage, and correlation of log data used in this work therefore correspond to established SIEM functionality and do not constitute an original contribution. OWASP A09 explicitly recommends centralised logging and monitoring and identifies solutions such as the ELK stack as possible tools for addressing logging and monitoring vulnerabilities [**OWASPA09**].

Therefore, the question changes from how events are analysed to how their underlying sources can be verified. In this context, Wazuh is particularly relevant, as it combines log analysis with file integrity monitoring and security configuration assessment. This addresses both event-based analysis and the verification of the system state [**WazuhDocumentation**]. However, these mechanisms mainly focus on configuration and file state rather than the relationship between configuration and individual runtime events. For instance, the security configuration assessment evaluates system configurations against defined policies, while file integrity monitoring detects changes to monitored files. Neither mechanism directly establishes whether a particular runtime event is consistent with the configuration state that was active when the event occurred.

A further concern arises when considering the integrity of the log sources themselves. Graves discusses this concern for Shibboleth under the STRIDE category of repudiation. Prompted by unusual entries reported on the Shibboleth mailing list, he concludes that log entries can be injected that lead the administrator astray, and suggests keeping the Apache logs, which contain the actual browser requests, separate from the Tomcat and IdP logs, which may themselves be subverted [**ShibbolethGIAC**]. CWE-117 describes the underlying mechanism: input that is written to a log without neutralisation allows an attacker to insert forged entries and thereby mislead subsequent analysis [**CWE117**]. An attacker who has gained access to the system can go further and alter entries that have already been written. OWASP A09 lists insufficient protection of logs against tampering as a weakness and recommends audit trails with integrity controls to prevent tampering or deletion [**OWASPA09**]. AppSensor dedicates a detection point to this issue (IE6), which involves modifying an existing log entry [**OWASPAppSensor**]. Snodgrass et al. assume such an attacker in their threat model, who after a successful penetration has access to the database, the operating system and the hardware [**TamperDetection**].

This makes the correlation of independent event sources more than a technical convenience. It provides an additional means of identifying inconsistencies between records of the same event. A system that relies solely on log analysis can compare multiple log sources, but if all sources are stored as files on the same system, they remain exposed to an attacker with sufficient file-system access.

Cryptographic approaches provide a stronger solution to this problem. Schneier and Kelsey, as well as Snodgrass et al., describe mechanisms that make tampering with stored logs cryptographically detectable [**CryptoSupportSecureLogs**, **TamperDetection**]. However, these approaches require changes to the logging or storage architecture and may depend on external trusted components. Snodgrass et al. report an overhead of up to 16 percent for their approach [**TamperDetection**].

The approach taken in this work is different. Instead of modifying the existing logging infrastructure, two independently generated records of the same event are compared. The protection therefore does not rest on cryptography, but on the temporal and architectural separation between the records. A log file may remain accessible to an attacker with file-system access for an extended period, whereas a record generated within the application process itself exists only for the duration of the corresponding processing before it is transmitted away. This does not make a record produced inside the application process immutable: an attacker controlling the JVM process could also manipulate it. Its purpose is instead to provide a second, independently generated observation of the same event.

The remaining question is whether such application-level detection is already sufficiently covered by existing frameworks. Of the works considered, AppSensor is conceptually closest to this work. However, its detection model is based on different assumptions about the monitored application [**OWASPAppSensor**].

AppSensor detection points primarily monitor the behaviour of a user within an application session. They can detect events such as tampered cookies, modified parameters, or injection attempts. The framework therefore asks whether a user's behaviour is suspicious. It does not provide a mechanism for comparing an observed runtime event with a configuration or security state declared by a third party.

This limitation follows from the architectural assumptions of the framework. AppSensor is designed for applications in which the application itself is the primary authority over the user's interaction. An IdP, in contrast, performs authentication and issues identity information on behalf of other entities. Consequently, security-relevant conditions may depend not only on the IdP's own runtime behaviour, but also on configuration and trust relationships involving its federation partners.

This creates a detection category that is not directly represented in the AppSensor model: comparing an observed runtime event against the configuration that should govern that event. One AppSensor detection point, however, is directly applicable. IE6, *Violation of Security Log Integrity*, addresses the integrity of security logs and cites NIST SP 800-92 and Snodgrass et al. as supporting references [**OWASPAppSensor**, **NIST80092**, **TamperDetection**]. The discrepancy-based approach used in this work therefore does not introduce the general concept of detecting log integrity violations. Instead, it applies an established detection principle to the specific domain of federated identity management and extends it with application-level observations from the Shibboleth IdP.

3.4 Derivation of the Rules

The derivation of the detection rules is based on three complementary sources: the FIM survey by Simpson and Groß, the OASIS SAML Security and Privacy Considerations, and the STRIDE threat classification scheme. The FIM survey identifies 11 vulnerabilities and 14 attack classes based on a systematic literature review [**FIMSurvey**]. The vulnerabilities are grouped into cross-protocol issues where the same weakness is exploited by the same attack class, results in the same CIA consequence, and is documented for at least two different protocols. The survey identifies 14 such issues. Six of them concern SAML

(issues 5, 7, 8, 9, 10 and 14 in the numbering of the survey), while one directly concerns Shibboleth (issue 6). The survey therefore provides the primary basis for identifying attack scenarios that are relevant to federated identity management.

While the survey describes vulnerabilities that have been documented in the literature, the OASIS SAML Security and Privacy Considerations provides security recommendations for SAML deployments by specifying security requirements and suggested countermeasures [**SAMLSecurityConsideration**]. This allows the detection rules to be derived not only from observed or documented attacks, but also from the security properties that a SAML deployment is expected to enforce.

A third perspective is provided by STRIDE, a threat classification scheme that categorizes threats according to their mode of operation [**ThreatModeling**]. It distinguishes between spoofing, tampering, repudiation, information disclosure, denial of service, and elevation of privilege. Graves applies this framework to the analysis of threats against Shibboleth [**ShibbolethGIAC**]. In this work, STRIDE is used to characterise the security impact of the attack scenarios identified by the other two sources.

The three sources therefore serve different purposes in the derivation process. The FIM survey identifies documented vulnerabilities and attack classes, OASIS specifies the security requirements that a SAML deployment should satisfy, and STRIDE provides a classification of the resulting security impact. Where the evidence from the survey and the requirements specified by OASIS overlap, the basis for a detection rule is particularly strong. Replay attacks provide an example of such an overlap, as they are documented in the literature and explicitly addressed by the SAML security considerations.

4 System Design

This chapter describes the design of the proposed monitoring system. It first derives the requirements that follow from the research questions and the preceding chapter, and then presents the system along the path that data takes through it. First, the data aggregation inside the IdP. Then, the transport of the collected data across a trust boundary. Finally, storage, correlation and rule-based detection.

The emphasis lies on the collection layer. The remaining layers follow established practice, whereas the question of which data can be obtained from within an IdP, and by which mechanisms, is specific to this work. Design decisions are stated together with the alternatives that were considered and rejected.

4.1 Requirements for the System

The requirements for the monitoring system were derived from the research and analysis of the related work. Section 3.3 showed that AppSensor's detection model provides no mechanism for comparing an observed event against a configuration declared by a third party. Since an IdP authenticates on behalf of other entities, such a comparison requires at least two independent sources. One source describing the intended configuration and another describing what actually occurred.

The classification in Section 3.2 placed this work at the *Monitor and Detect* level, with *Plan and Prevent* deliberately excluded. The system must therefore report detected security-relevant events without automatically intervening.

As mentioned in Section 3.2, Hall et al. identify the integration of detection points into the application as a central difficulty [**BlackWatch**]. The system is therefore implemented as a plugin that uses an extension point provided by Shibboleth, allowing the monitoring functionality to be added without modifying the underlying IdP implementation.

Section 3.3 also showed that log sources stored on the same system share the same attack surface. Records generated by the interceptor are therefore transmitted to a separate system rather than stored locally. This requires the transport to be authenticated and sensitive data to be pseudonymised at the source.

The rules presented in Section 3.4, derived from the FIM survey by Simpson and Groß, the OASIS SAML Security and Privacy Considerations, and the STRIDE threat classification scheme, provide the actual detection logic for identifying unusual authentication behaviour and security-relevant conditions within the federated identity environment.

These considerations result in five requirements:

1. Two independent sources: the declared configuration and the observed event.
2. Detection and reporting without automated intervention.
3. Realisation as a plugin, without modification of the IdP.
4. Storage outside the monitored system, with authenticated transport and pseudonymisation at the source.

5. Detection rules derived from documented attack classes and security recommendations.

4.2 Overview

The proposed system comprises four layers. The data collection layer gathers security-relevant information from within the Shibboleth IdP, drawing on three sources: the audit log written by the IdP, its static configuration and metadata files, and two custom interceptors that are attached at two points of the profile processing and execute inside the IdP itself. Because the monitoring backend resides outside the trust boundary of the IdP, sensitive attributes such as usernames and IP addresses are pseudonymised before transmission. The transport layer then transfers the collected records to the backend application over an authenticated channel. Within the backend, incoming records are persisted and periodically correlated, so that observations originating from different sources but describing the same authentication event can be linked. The detection layer evaluates the resulting data against the rule catalogue in order to identify configuration weaknesses and anomalies in the authentication process. Finally, the presentation layer makes the detected findings accessible to the operator through a frontend web application. Figure 4.1 illustrates the UML deployment diagram of the proposed system.

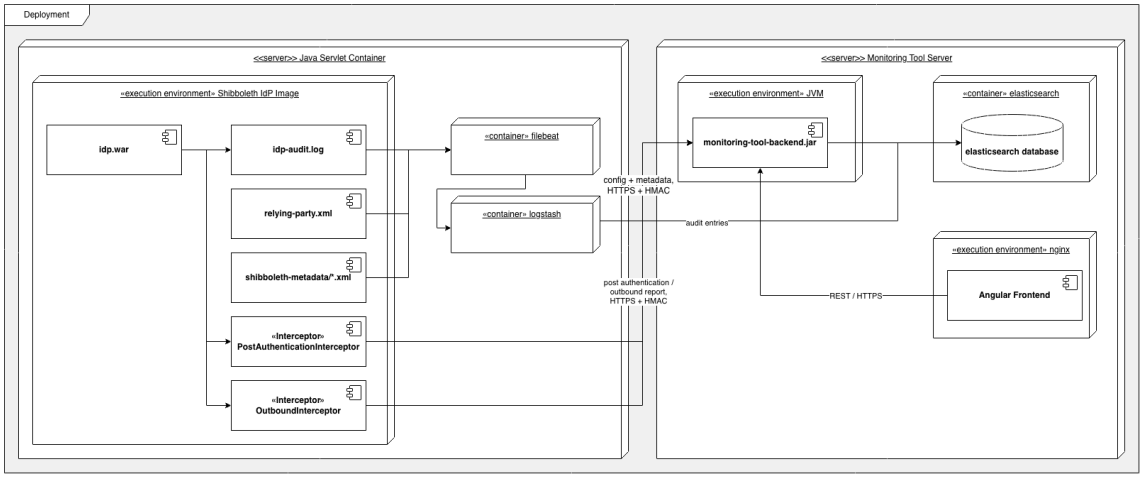


Figure 4.1: *UML Deployment Diagram of the Proposed Monitoring System*

Two terms recur throughout this work. An audit entry describes a record that the IdP writes to its audit log and that reaches the monitoring tool through the log shipping stack. A report represents a record that an interceptor produces within the IdP and transmits directly to the monitoring tool. Since two interceptors are used, their reports are called the post authentication report and the outbound report where they have to be distinguished. A report and an audit entry may describe the same authentication event, which makes the correlation described in Section 4.5.1 possible.

4.3 Data Aggregation Layer

The design of the collection layer begins with a broader question than the choice of individual data sources. Through which kinds of channel does an application disclose information about itself at all? Applied to the Shibboleth IdP, five such channels can be identified. The application writes log files, it reads static configuration and metadata, it maintains runtime state that is accessible through defined extension points, it exposes status endpoints, and it publishes metrics via Java Management Extensions.

Not all of these are suitable for the purpose of this work. The system described here associates every finding with an individual authentication event, since a security relevant condition is only meaningful in the context of the transaction in which it occurred. A channel that cannot be related to a single login can therefore not contribute to correlation or to the generation of an alert.

Two of the five channels fail this criterion. Status endpoints report the state of the deployment as a whole, such as its version, uptime, configured metadata providers or the status of the underlying services. They therefore address availability rather than the security of individual transactions. JMX metrics are aggregates by construction. A counter reports that a number of logins occurred, not which login was affected. An alert derived from either source could not be attributed to a transaction.

The remaining three channels form the collection layer of this work. These are the audit log, the configuration files, and the runtime state accessible through interceptors. They are described in the following sections.

4.3.1 The Audit Logs

All logging within the Shibboleth IdP is performed through the abstract logging API SLF4J, with Logback as the default implementation. By default, three classes of log files are produced. The diagnostic logs, *idp-process.log* and *idp-warn.log*, contain messages generated during the normal operation of the software. The general audit log, *idp-audit.log*, records the results of authentication transactions for each SP. Finally, the consent audit log, *idp-consent-audit.log*, contains audit records of user decisions regarding data privacy and terms of use [**LoggingConfiguration**].

The amount of information recorded in the diagnostic logs depends on the configured logging level, which determines the severity of messages that are written. The *idp-warn.log* file is a filtered view of *idp-process.log* and contains only messages logged at the `WARN` level or higher. These files are written for a human reader. A line consists of a timestamp, a severity, the originating class and a free-form message whose content depends on the code location that produced it, so that lines of different origin share no common structure.

The audit log differs from the diagnostic logs in that its structure is defined rather than free-form. It records one entry per completed transaction, written once the IdP has finished processing the request, with a fixed sequence of fields that allows every line to be decomposed in the same way. This has two consequences for the information it can provide. On the one hand, the entry reflects the final outcome of a transaction, including properties that are only determined at the end of processing, such as the encryption applied to the

outgoing assertion. On the other hand, events that never result in a completed transaction produce no entry. Failed password validations in particular are not recorded in the audit log by default, since no transaction is concluded for them.

As mentioned in Section 2.3.3, the fields that are written to the audit log are not fixed. The audit format is configured per profile in *conf/audit.xml*, where each field is represented by a formatting token, and the set of available tokens can be extended within the IdP [**AuditLoggingConfiguration**]. Which information a given deployment records is therefore determined by its operator and may differ between installations.

This property has direct implications for the monitoring system developed in this work. The system cannot assume the presence of a fixed set of audit fields, since the information required by a particular detection rule may simply not be recorded. The detection logic must therefore take the configured audit format into account and determine whether the information a given rule depends on is available at all. The same constraint applies to the correlation between audit entries and interceptor records. Rather than relying on a fixed set of fields, the correlation must adapt to the information that a given deployment provides, which requires a strategy capable of selecting an appropriate matching approach depending on the fields available for a given audit event.

The audit format of the deployment was extended by the identifier and the issue time of the response and by the identifier of the request that it answers. These tokens make the response level fields that the outbound interceptor reads comparable with the audit entry, as described in Section 5.1.

4.3.2 Metadata and Relying Party File

In a Shibboleth IdP, SAML metadata describes the entities that participate in a federation and provides the information required for establishing communication between them. Metadata is represented in XML and contains information about the roles, endpoints, supported protocols, and cryptographic keys of the participating entities [**Metadata**].

A SAML entity can support one or more roles, which are represented by role descriptors within the metadata. For SPs, the relevant role is represented by the `md:SPSSODescriptor` element. This descriptor contains information required by the IdP to communicate with the SP, including the supported SAML protocol families specified by the `protocolSupportEnumeration` attribute, cryptographic keys, supported name identifier formats, and the endpoints provided by the SP [**MetadataForSP**]. A central component of an SP's metadata is the `md:AssertionConsumerService` element. It specifies an endpoint to which the IdP can send SAML responses and assertions on behalf of the SP. By explicitly enumerating these endpoints in the metadata, the IdP can determine the locations to which SAML messages may be sent [**MetadataForSP**].

The Shibboleth IdP consumes metadata through configured metadata providers. These providers define the sources from which SAML metadata is obtained and are configured using `<MetadataProvider>` elements in *conf/metadata-providers.xml* [**MetadataConfiguration**]. The metadata therefore forms an essential part of the IdP configuration, as it determines which federation entities are known to the IdP and which endpoints, protocols, and cryptographic information are associated with them.

Alongside the runtime information recorded in the audit log, the IdP holds a second class of information that describes not what happened, but what is supposed to happen. This information is contained in the configuration files of the deployment, most notably in *relying-party.xml*.

This file specifies which functional features the IdP supports and allows these settings to be customised based on the identity of a relying party. It determines which profiles are enabled for a given partner and allows security configurations to be attached at various levels. These include whether assertions and other content are encrypted, at which layer signing is performed, and which cryptographic algorithms are used [**RelyingPartyConfiguration**].

Of particular relevance to this work is that these settings are not global. A deployment defines default behaviour that applies to all partners, and may then override this behaviour for individual relying parties or groups of them. A single IdP therefore enforces a set of independent, per partner policies rather than one uniform policy, and the security properties of a given transaction depend on which policy applies to the SP involved.

Two settings are relevant in the context of this work. The first concerns encryption. A deployment may disable the encryption of assertions for a particular partner, for instance where an SP does not support it, which means that assertions for that partner are transmitted in plain text. The second concerns authentication. A deployment may specify which authentication flows are permitted for a partner, and thereby require multi factor authentication for some SP while accepting password based authentication for others.

Both settings describe a target state. Whether a given login actually satisfied it is recorded in the audit log, not in the configuration. Neither source is sufficient on its own. A comparison between them is only possible if both are available to the monitoring system, which is the reason why the configuration is treated as an independent data source in this work.

Finally, *relying-party.xml* is also the file in which interceptor flows are enabled for a profile. The extension point used by the collection mechanism described in Section 4.3.3 is therefore configured in the same place as the security settings it observes.

4.3.3 Interceptor

As mentioned in Section 2.3.1 Shibboleth provides three interception points within its profile processing: inbound message processing, post authentication processing, and outbound message processing. Interceptors are implemented as Spring Web Flow subflows that are executed at these predefined points without requiring modifications to the underlying profile flow [**ProfileHandling**, **ProfileInterceptConfiguration**].

Inbound interceptors are executed during the processing of an incoming request. At this point, the inbound `MessageContext` tree is generally populated and, where applicable, contains a `RelyingPartyContext` as well as contexts required for operations such as transport or message authentication [**ProfileHandling**].

Consequently, an inbound interceptor can observe information associated with the received message and the context in which it is being processed. This makes the interception point particularly suitable for security checks on the incoming request. Shibboleth itself uses inbound interceptors for various security policy checks. An interceptor may also

modify the context tree, although such modifications directly influence subsequent profile processing [**ProfileHandling**].

For the monitoring system developed in this work, the inbound interception point is therefore relevant for observing properties of the incoming SAML message and its processing context before the main profile execution continues.

The post authentication interception point is executed after authentication and attribute lookup have taken place, but before the profile constructs the outgoing message. Unlike the inbound and outbound hooks, this interception point is only available for a subset of profiles, particularly profiles that perform user authentication [**ProfileHandling**].

The interceptor therefore has access to a later state of the processing context. In particular, information related to the authenticated identity and resolved attributes is available at this stage. The interceptor can also modify the context before the outgoing response is constructed, including changes to the authenticated identity or attributes [**ProfileHandling**].

This makes the post authentication interception point particularly suitable for the monitoring system developed in this work, as it provides access to authentication related information that is not yet available at the inbound stage, and it does so before the profile constructs the outgoing message.

Outbound interceptors run shortly before the final stage of profile processing that produces the response to the client. Their input therefore represents the state produced by the preceding profile execution and typically contains a populated outbound `MessageContext`. The message may not yet be in its final form, since operations such as signing can still occur afterwards [**ProfileHandling**].

The outbound interception point consequently provides access to the information that is about to be returned to the relying party. Interceptors can inspect or modify the outbound context and may also terminate processing [**ProfileHandling**]. For the monitoring system, this interception point is useful for observing the result of the authentication workflow and properties of the response that is about to be issued by the IdP.

The monitoring system uses two interception points, the post authentication point and the outbound point. An inbound interceptor would see the incoming message but not the outcome of authentication, since that has not yet taken place. The post authentication point is the earliest point at which both the incoming message and the authentication result are available in the same context, and it serves as the main collection point. The outbound point still has access to both, since its input is the entire state of the tree that the profile execution has produced [**ProfileHandling**], and it additionally holds the response that is about to be issued. At the post authentication point, in contrast, the outgoing response does not exist yet. The outbound interceptor therefore complements the main collection point with the fields that only the response carries. The outbound point is also reached by responses that the IdP issues without authenticating the user, for instance an error response for an SP whose configuration cannot be fulfilled, which the post authentication point never sees.

What remains outside the view of the post authentication point is the outgoing response, including the response identifier, the final status of the transaction and the encryption actually applied. The outbound interceptor makes these properties observable, with one limit. When the assertion is encrypted, the encryption has already taken place when the

outbound hook runs, so that the assertion identifier, the authentication context and the name identifier are no longer readable, and the IdP cannot read them afterwards, since only the SP holds the private key. Reading them before the encryption would require an intervention in the profile flow itself, which the third requirement from Section 4.1 excludes. For encrypted assertions only the response level fields and the encryption state are therefore available to the interceptors, whereas the audit log records all of them, since the IdP extracts them at an earlier point of the processing sequence. The complementarity of the sources described in Section 4.3.1 therefore follows from their position in the processing sequence rather than from a deliberate division of responsibilities.

4.4 Transport Layer and Trust Boundary

After the data has been aggregated on the Shibboleth IdP side, it is transferred to the monitoring tool for further processing and storage. As stated in Section 4.2, this tool is deliberately deployed outside the trust boundary of the IdP. The separation avoids placing the monitoring system under the same security assumptions and access controls as the system it monitors.

If the monitoring functionality were deployed entirely within the IdP, an attacker who compromises the IdP could potentially compromise or manipulate the monitoring functionality at the same time. An attacker would then be in a position to influence the recording of their own activity, since collection, storage and analysis would all fall under the same control.

By transferring the collected information to a separate monitoring component, the observation and analysis functions are separated from the monitored application. This creates an additional security boundary and reduces the dependency of the monitoring system on the integrity of the IdP itself. The external component therefore does not exist solely for scalability or architectural convenience. What the separation protects is the data that has already been transmitted. An attacker who gains control of the IdP can influence which records are produced from that point onward, but not those that have already left the system. The independence is thus temporal rather than absolute.

This design decision has consequences. The collected data leaves the security domain in which the original authentication data is processed and becomes accessible to a separate component with a different trust relationship. Therefore, transmitting original sensitive user information, such as usernames and client IP addresses, would expose more personal information than is necessary for the intended processing purpose. A comparable concern is described by CWE-532 for log files, where the inclusion of sensitive user data provides attackers with an additional and less protected path to obtaining that information [**CWE532**]. The same reasoning applies to data transferred to a component outside the original security domain.

To reduce this exposure, sensitive identifiers are pseudonymised before they leave the IdP trust boundary. The monitoring functionality does not require the original identity of a user to perform its security analysis. It primarily requires the ability to distinguish and correlate events belonging to the same subject. The monitoring system can therefore

preserve the ability to correlate multiple events belonging to the same subject without transmitting the original identifier. Pseudonymisation reduces the risk associated with unauthorised access to the monitoring system, while still allowing the required security analysis to be performed.

To enable this correlation across the different data sources, all components that generate monitoring data must apply the same pseudonymisation algorithm and the same secret salt. Otherwise, identical identifiers would result in different pseudonyms and events originating from different sources could no longer be reliably correlated. The secret salt is therefore treated as a shared cryptographic parameter between the data collection components and must be protected accordingly.

The deployment of the monitoring tool outside the IdP trust boundary introduces an additional security concern. The endpoints of the monitoring tool accept data from collection components that reside outside its own trust boundary. Consequently, the monitoring tool cannot assume that every received message originates from an authorised data collection component.

Without message authentication, an attacker could submit forged or manipulated records to the exposed endpoints. Injected events would not only produce false findings but could also interfere with the correlation of genuine ones, since a fabricated record may be matched against a real audit entry.

The transmitted data must therefore be authenticated before the monitoring tool accepts it. In this work, this is achieved using a Hash-based Message Authentication Code (HMAC) [**HMAC**]. The collection components and the monitoring tool share a secret key that is used to generate and verify a message authentication code for each transmitted message.

The final design decision concerning the transport layer is whether the collection component should wait for the monitoring tool to confirm receiving the data. The interceptors execute within the authentication flow of the IdP. Any time spent transmitting a record is therefore time added to the login of the user who triggered it. Waiting for a response would make the duration of every authentication dependent on the availability and response time of a component outside the IdP, and a monitoring tool that is slow or temporarily unreachable would delay the service it observes.

The records are consequently transmitted asynchronously. Each interceptor dispatches its record and continues processing without awaiting a response. This has a cost. If a transmission fails, the record is not sent again and is missing in the monitoring tool, and only a warning in the log of the IdP indicates the failure. The design therefore prioritises non-interference with the monitored service over a guarantee of delivery. For a monitoring system that observes a central authentication service, this trade-off appears justified. A monitoring component that degrades the availability of the service it observes would undermine its own purpose. Hall et al. decouple analysis for a similar reason: responses are retrieved separately rather than returned in reply to the submitted event [**BlackWatch**].

4.5 Correlation and Detection Layer

This section explains how the monitoring tool processes the collected data. Records originating from different sources are first linked to the authentication event they describe, then evaluated against the detection rules, and finally classified according to the impact of the condition they report.

4.5.1 Correlation

When the collected data arrives at the monitoring tool, it must first be stored for subsequent processing. The different data sources transmit their information through separate endpoints and at different times, which means that information belonging to the same authentication event does not necessarily arrive together. The corresponding records can therefore not always be correlated upon arrival. Instead, they are stored until the required information from the other source becomes available.

This introduces the first challenge for the correlation process. Data collected by the post authentication interceptor within the Shibboleth IdP is transmitted immediately after the collection process has completed, so the corresponding post authentication report reaches the monitoring tool shortly after the authentication event has been processed. The audit entry follows a different path. Audit logs are collected by log shippers that operate periodically, and the corresponding entry therefore reaches the monitoring tool later than the post authentication report. This temporal offset is the reason why the two records cannot be matched by a simple database join at the time of arrival.

A further consequence follows from this. A report may arrive at a point when its counterpart does not yet exist, which means that an unsuccessful correlation attempt does not necessarily indicate that no counterpart will ever arrive. The correlation process must therefore retry over a defined period and, once that period has elapsed, treat the record as permanently uncorrelated. Without this distinction, a record still awaiting its counterpart would be indistinguishable from one that will never receive it, and the proportion of successfully correlated records could not be determined.

The second challenge arises from the dynamic nature of the audit log discussed in Section 4.3.1. Since the audit format is determined by the configuration of the deployment, the information available in an audit record is not fixed, and the correlation must adapt to the fields that are actually present.

This creates an asymmetric correlation problem. The data collected by the post authentication interceptor is static by design and always contains the same set of fields. The audit data, in contrast, is dynamic and may lack information required by a particular correlation strategy. The static post authentication report, which arrives first, therefore has to be matched against a dynamic audit record that arrives later.

Depending on the fields available in the audit record, different correlation strategies are applicable, and these differ in how reliably they identify the corresponding record. A strategy based on a unique identifier that both sources share establishes the relationship unambiguously, whereas a strategy based on attributes that several transactions may have in common can only approximate it. The correlation process consequently applies the

available strategies in a defined priority order, using the strongest applicable strategy first and weaker ones only where the required fields are absent. The strategy used for a given record is recorded alongside the result, since it determines how reliable the resulting correlation is. The precision achieved by each strategy is examined in Chapter 6.

4.5.2 Detection

Once the data has been collected and stored, it is evaluated against a catalogue of detection rules, where possible. With two exceptions described below, a rule receives a single object as input and either returns a finding or nothing. The conditions are stated explicitly rather than learned from data, which means that every finding can be traced back to the condition that produced it and, through the derivation in Section 3.4, to the documented weakness or requirement it is based on.

The rules do not all operate on the same kind of input. Four input types occur in this system: login events derived from the audit log, the configuration of an SP, its metadata, and the post authentication reports produced by the interceptor. These differ not only in structure but also in the rhythm at which they appear. Login events occur continuously during operation, whereas a configuration or a metadata document changes rarely and may remain unchanged for long periods of time.

Evaluating all rules against all data whenever any of it changes would therefore be inefficient and, in the case of configuration data, largely redundant. The detection logic is consequently divided into four engines, each responsible for one input type. An engine evaluates the rules that apply to its input and can be scheduled according to the frequency at which that input actually changes.

A further consideration follows from the dynamic audit format described in Section 4.3.1. A rule may depend on a field that a given deployment does not record, in which case the rule cannot reach a valid conclusion. Evaluating it regardless would produce a result that appears definite but rests on absent information, and a rule that cannot observe a condition must not report its absence as a finding. Each rule therefore declares which fields it requires, and the engine verifies their availability before evaluating the rule. Where the required information is not recorded, the rule withdraws rather than returning a result. Which rules are active in a given deployment is thus determined by what that deployment logs.

Which conditions the rules examine is a separate question from how they are evaluated. Thirteen of the fifteen rules can be traced to documented sources. Ten of them were derived from the three frameworks introduced in Section 3.4, namely the survey by Simpson and Groß, the OASIS security considerations and the STRIDE classification. `FAILED_LOGIN` reports an indicator rather than an attack and follows the OWASP guidance on security logging. The two rules that compare independently produced records follow the log integrity argument of Section 3.3. The remaining two rules cannot be traced and are discussed after Table 4.1, which shows the resulting assignment for each rule.

Two observations follow from this assignment. Lack of binding appears in three of the six SAML related entries and is therefore the most frequently documented weakness of the protocol in the survey. Both replay rules address it, although only a replayed

Table 4.1: *Assignment of the 15 Detection Rules to Documented Vulnerabilities and Attack Classes*

Rule	Vulnerability	Attack Class	STRIDE	Source
WEAK_ENCRYPTION	Unencrypted Communications	Message Modification	I, T	[**FIMSurvey**, **BSITR02102**, **SAMLSecuri**
UNENCRYPTED_ ASSERTION	Unencrypted Communications	Message Modification	I, T	[**FIMSurvey**, **ShibbolethGIAC**]
WEAK_ CONFIGURATION	Unencrypted Communications	Message Modification	I, T	[**FIMSurvey**]
MISSING_MFA	Weak User Credentials	Enables Brute Force	S, E	[**FIMSurvey**]
MFA_BYPASS	Weak User Credentials	Enables Brute Force	S, E	[**FIMSurvey**]
BRUTE_FORCE	Weak User Credentials	Brute Force	S, E	[**FIMSurvey**]
FAILED_LOGIN	Weak User Credentials	Indicator, not an attack	S	[**OWASPA09**]
ASSERTION_REPLAY	Lack of Binding	Replay Attack	S, E	[**FIMSurvey**, **SAMLSecurityConsiderati**
REQUEST_REPLAY	Lack of Binding	Replay Attack	S, E	[**FIMSurvey**, **SAMLSecurityConsiderati**
SIGNATURE_ WRAPPING_ SUSPECTED	Message Formatting	Message Modification	S, T, E	[**BreakingSAML**, **ShibbolethGIAC**]
CERTIFICATE_ EXPIRED	Security recommendations	None	S	[**SAMLSecurityConsideration**]
CORRELATION_ DISCREPANCY	Log integrity	Log tampering	T, R	[**OWASPAppSensor**, **OWASPA09**, **CWE1**
FIELD_MISMATCH	Log integrity	Log tampering	T, R	[**OWASPAppSensor**, **OWASPA09**, **CWE1**
SUSPICIOUS_USER_ AGENT	None	None	—	—
BEHAVIOUR_ANOMALY	None	Detection method	S	—

request passes the IdP, whereas a replayed assertion is presented to the SP (Section 7.2). Furthermore, several rules form pairs in which the same weakness is captured twice, once as a configuration state and once as an event that has occurred. MISSING_MFA examines whether an SP requires multi factor authentication at all, whereas MFA_BYPASS examines whether an actual login circumvented that requirement. This distinction is a direct consequence of the three source architecture described in Section 4.3: the configuration states what is intended, and the audit log records what took place.

Two of the rules do not examine a single record. They compare records that were produced independently of each other for the same event. The first, CORRELATION_ DISCREPANCY, reports a post authentication report for which no audit entry arrived within the retry window of the correlation. The second, FIELD_MISMATCH, compares the fields of the outbound report with those of the audit entry that the post authentication report of the same login was correlated with, and reports every field in which they disagree. Both follow the detection point IE6 of AppSensor, the violation of security log integrity, and the argument of Section 3.3 that two independent records offer an alternative to protecting a

log cryptographically [**OWASPAppSensor**]. In addition, input that is written to the audit log without neutralisation can corrupt individual fields of an entry or split it into several lines, which the comparison with an independently produced record reveals as a field mismatch or, if the entry can no longer be correlated, as a discrepancy [**CWE117**].

The comparison establishes that two records disagree but not which of them was altered, since an attacker who controls the application process can alter the report as well. It also cannot compare fields that only one of the records contains. For encrypted assertions this applies to the fields inside the assertion, which the outbound interceptor cannot read after the encryption and which only the audit log records. A field is compared only if the interceptor observed it and the audit format of the deployment records it, in line with the rule of the other detectors that a rule should not guess when the data it requires is absent.

Two rules cannot be assigned to any of the three frameworks. `SUSPICIOUS_USER_AGENT` identifies an indicator for which no documented relation to a specific attack class could be established. `BEHAVIOUR_ANOMALY` is a detection method rather than an attack class, since it may reveal several different attacks indirectly without corresponding to any one of them. Both rules are retained in the catalogue, and the absence of a derivation is stated here rather than omitted.

4.5.3 Severity Classification

Each alert detected by the detection layer described in Section 4.5 is assigned a severity level. The severity levels used in this work are based on the high-level risk categories proposed by Graves, which range from R1 to R5. Although Graves does not state an explicit ordering, he identifies the theft or abuse of the signing keys underlying R5 as the worst case. This work treats the categories as increasing in severity. The specific definitions of the risk categories are as follows [**ShibbolethGIAC**]:

- **R1:** Abuse of system resources
- **R2:** Log on to a specific SP as any user
- **R3:** Log on to a specific SP as a specific user
- **R4:** Steal user passwords
- **R5:** Log on to any SP as any user

This classification describes the potential impact of an attack from an attacker's perspective. However, the rules developed in this work identify two different types of security conditions. Some rules detect vulnerabilities that can enable an attack, while others detect attacks or anomalous events that have already occurred. Severity classification is therefore performed in two steps. First, the identified risk determines the base severity level. Second, the classification is adjusted according to whether the rule identifies only a condition that enables an attack or evidence that the corresponding event has already occurred. A detected attack is therefore assigned a higher severity than a vulnerability that merely enables the attack.

The classification serves as an orientation rather than as a formula. The severity is therefore assigned per rule, as listed in Table 4.2, and it is raised or lowered only where the impact of a rule depends on its context. The two multi factor rules are critical if the SP

is on the list of providers that must use multi factor authentication, and MFA_BYPASS also if the configured flow requires it in every case, and medium otherwise. WEAK_ENCRYPTION takes the severity from the algorithm policy, CERTIFICATE_EXPIRED from the expiry status of the certificate, and BEHAVIOUR_ANOMALY is rated high if at least two independent reasons apply and medium otherwise.

The discrepancy rule is classified as medium, since a missing counterpart can also result from operational causes such as a delayed log shipping chain. The mismatch rule is classified as high, since a contradiction between two records of the same event points either to a manipulation or to a defect in the collection, and both undermine the basis on which the other rules operate.

Table 4.2: *Severity assigned by each rule*

Severity	Rules
Critical	UNENCRYPTED_ASSERTION, WEAK_CONFIGURATION, ASSERTION_REPLAY, SIGNATURE_WRAPPING_SUSPECTED
High	BRUTE_FORCE, REQUEST_REPLAY, FIELD_MISMATCH
Medium	CORRELATION_DISCREPANCY
Low	FAILED_LOGIN, SUSPICIOUS_USER_AGENT
Context	MISSING_MFA, MFA_BYPASS, WEAK_ENCRYPTION, CERTIFICATE_EXPIRED, BEHAVIOUR_ANOMALY

4.6 Visualisation Layer

The primary purpose of the visualisation layer is to present the collected, correlated, and detected information to an operator through a user interface. The layer does not generate new information. Instead, it presents the data produced and aggregated by the preceding layers.

Accordingly, the interface provides visibility into detected findings but does not offer any functionality for modifying or controlling the IdP. This is a deliberate design decision rather than an omission and implements the second requirement described in Section 4.1, which follows from the passive monitoring principle adopted in this work.

Due to the pseudonymisation of user identifiers, the interface does not display plaintext identities. The monitoring system only retains pseudonymised identifiers and can therefore determine whether multiple events originate from the same user without revealing that user's actual identity. An operator who requires the original identity would have to obtain it from the IdP itself. Data protection is thus achieved through the architecture of the system rather than through access control alone.

The findings generated by the detection layer described in Section 4.5 are assigned severity levels according to the impact that a successful exploitation of the underlying weakness would have. The derivation of these levels is described in Section 4.5.3. They allow the operator to distinguish between findings of different importance and support the prioritisation of security relevant events.

In addition to the finding itself, the interface indicates which correlation strategy was used to link the underlying records. As described in Section 4.5, the available strategies differ in how reliably they establish the relationship between a post authentication report and an audit entry. Making this information visible allows an operator to judge how well supported a given finding is, rather than presenting all findings as equally certain.

4.7 Design Decisions and Rejected Alternatives

This section presents the main design decisions made for the monitoring tool and discusses the alternatives that were considered but rejected. Each decision involves a trade off, and the cost of the chosen option is stated alongside its rationale.

The first alternative was to extend the monitoring tool with the capability to actively intervene in ongoing attacks. In this design, the tool would provide the operator with the option to react to a detected finding by terminating the affected user session or by blocking the corresponding SP. This design was rejected because an incorrect response would not necessarily affect only the service concerned. Since the IdP provides authentication for multiple SPs, an erroneous intervention could disrupt several or even all connected services. The monitoring system is therefore intentionally designed as a passive system. The cost of this decision is that the system cannot prevent an attack that is in progress. It reports the condition it observes and leaves any reaction to the operator.

The second alternative was to incorporate cryptographic mechanisms for detecting log manipulation, following the approaches of Schneier and Kelsey and of Snodgrass et al. discussed in Section 3.3 [**CryptoSupportSecureLogs**, **TamperDetection**]. Such mechanisms can provide cryptographic evidence of modifications to stored log data. Their integration would, however, require changes to the existing logging and storage infrastructure and would introduce additional processing overhead, which Snodgrass et al. report as up to 16 percent [**TamperDetection**].

The proposed system therefore does not provide cryptographic proof of log integrity. Instead, it detects potential inconsistencies by comparing audit log entries with information independently collected by the interceptors. This avoids modifying the existing logging infrastructure while still providing a second observation of the same authentication event. The cost is that an inconsistency indicates a discrepancy between two sources without establishing which of them was altered.

The third alternative was to use machine learning instead of explicitly defined detection rules. In such an approach, the system would learn normal behaviour statistically and identify deviations from it as anomalies. This approach was rejected because the detection rules developed in this work are explicitly derived from documented vulnerabilities, security requirements, and threat classifications. A rule based approach therefore provides direct traceability from a finding to the underlying security rationale, which is a central objective of the derivation described in Section 3.4. The cost of this decision is that the system detects only conditions that were anticipated when the rules were formulated. Attack patterns that do not correspond to any defined rule remain unnoticed, whereas a statistical approach could in principle identify previously unknown deviations.

A further decision concerns the number of interceptors. The system collects at two points, since the post authentication point was the basis of the collection and the outbound point was added to make the fields of the response observable. The outbound point turned out to see more events than the post authentication point. For an SP that was configured for encryption without a suitable key, the IdP answered with an error status before authentication began, as the empty subject and authentication fields of the audit entry show, and only the outbound interceptor produced a report. In principle, a single outbound interceptor could collect the data of both interceptors and halve the number of transmissions per login, since the contexts read by the post authentication interceptor are still part of the context tree at the outbound point. The assertion level fields of encrypted assertions would remain unavailable, as they are with the two interceptors used here. This was not examined beyond the request identifier, which the outbound interceptor already reads from the inbound message context.

Two further decisions have already been discussed in the context in which they arise. The choice of the interception points, described in Section 4.3.3, determines which information the collection mechanism can observe and which remains outside its view. The decision to transmit records asynchronously, described in Section 4.4, prioritises non interference with the monitored service over a guarantee of delivery.

5 Implementation

This chapter describes the implementation of the proposed monitoring tool and discusses the main challenges encountered during development. It follows the same structure as Chapter 4 by examining each system layer and describing its technical implementation.

5.1 Data Aggregation Layer

As described in Section 4.3, the data aggregation layer uses log shippers to collect and parse log entries and static configuration files, as well as two custom interceptors that operate within the Shibboleth IdP, the post authentication interceptor and the outbound interceptor, which are named after the hook at which they run. The log shipping stack consists of Filebeat for reading the source files and Logstash for preprocessing and transmitting the collected data. Both components run as separate containers on the same host as the IdP and access its log files and configuration through mounted directories. This placement is what allows them to share the pseudonymisation salt and the message authentication secret with the interceptor, as described in Section 5.2.

A key aspect of the implementation is the handling of audit log entries. Instead of parsing the audit log according to a fixed format, Logstash processes the audit fields as key-value pairs. This provides the technical basis for adapting to changes in the configured audit format and enables the monitoring system to handle the dynamic structure of the audit logs.

The monitoring tool handles these dynamic changes by maintaining a field registry. The registry periodically retrieves the most recent audit entries stored in the database and extracts the names of all fields that are currently present. Using only recent entries ensures that fields no longer written by the IdP are eventually removed from the registry. Both the detection rules and the correlation strategies query the registry before they are applied, which is how the withdrawal described in Section 4.5.2 is realised in practice.

The audit log is not the only log file read by the collection layer. As described in Section 4.3.1, authentication attempts that do not result in a completed transaction produce no entry there, which applies in particular to failed password validations. In the deployment used in this work, this authentication auditing is enabled and its records are routed to separate files, *idp-password-audit.log* and *idp-mfa-audit.log*, which record the outcome of the corresponding authentication flows. Filebeat is configured to read these files in addition to the general audit log, and the Logstash pipeline tags the resulting records with their source so that they remain distinguishable in the backend. Without this second source, the rules that evaluate failed authentication attempts would have no data to operate on.

A particular difficulty of this mechanism is that a missing field does not manifest itself as an error. If the audit format of a deployment does not include a field that a rule depends on, no exception is raised and no entry is written to any log. The information is simply absent, and a rule evaluating it would silently operate on incomplete data. The

registry therefore does not only serve adaptability, but also makes this absence explicit and detectable.

The static configuration files are collected by the same log shipping stack, but processed differently. Unlike the audit log, which is written continuously and line by line, *relying-party.xml* and the metadata documents change rarely and are read as a whole. Filebeat is therefore configured to aggregate the lines of such a file into a single event before transmitting it, and Logstash forwards the result without field level parsing. The number of lines combined in this way is bounded by configuration, which places an upper limit on the size of a document that can be transmitted as a unit. The interpretation of the XML structure takes place in the monitoring tool itself, where the relevant security settings are extracted and stored per SP. Filebeat monitors these files and transmits them only when their content has changed. The backend therefore does not receive the same configuration repeatedly, but only when a deployment has actually been modified, which is also what makes the detection rules operating on configuration meaningful at all.

Unlike the log shippers, the interceptor is a custom component developed as part of this work. It is registered as a Spring bean within the Shibboleth IdP and activated through the `postAuthenticationFlows` property of the corresponding profile configuration [**ProfileInterceptConfiguration**]. The post authentication interceptor therefore integrates into the IdP without any modification of its source code, which is how the third requirement of Section 4.1 is met.

The post authentication interceptor extends the `AbstractProfileAction` API of the Shibboleth IdP and operates on the `ProfileRequestContext`, which represents the current state of the profile execution [**AbstractProfileAction**, **ProfileRequestContext**]. From this context it accesses the child contexts that carry the information relevant to the monitoring purpose. The `RelyingPartyContext` identifies the SP for which the authentication is performed, the `SubjectContext` provides the authenticated subject, and the `InboundMessageContext` gives access to the raw SAML request as it was received [**RelyingPartyContext**, **SubjectContext**, **InOutOperationContext**]. Further contexts, among them the `AuthenticationContext` and the `SessionContext`, supply information about the authentication flow that was used and the session state at the time of the request [**AuthenticationContext**, **SessionContext**].

The outbound interceptor is implemented as a second action class with its own Spring Web Flow subflow, which is registered next to that of the post authentication interceptor in the intercept configuration of the IdP. It is enabled for an SP in *relying-party.xml* through the *outboundInterceptorFlows* property of the profile configuration, in addition to the *postAuthenticationFlows* property of the post authentication interceptor. The flow contains a single state that calls the action and then proceeds, so that the processing of the profile is not influenced [**ProfileInterceptConfiguration**].

The action reads the outbound message context. If it does not contain a SAML 2 response, the action returns without effect. Otherwise it collects the status code, the identifier and the issue time of the response, the identifier of the request that the response answers, and whether the response contains encrypted assertions. If the assertion is not encrypted, it additionally reads the identifier and issue time of the assertion, the authentication instant, the session index, the authentication context class and the format

and value of the name identifier, of which the value is pseudonymised before transmission in the same way as the subject and the address in the post authentication report. For encrypted assertions these fields are not collected, since the assertion is only available as ciphertext at this point and the IdP holds no key to decrypt it. The destination attribute of the response is empty at this hook and is therefore not used. The identifier of the original request, which links the outbound report to the post authentication report, is read from the inbound message context, which is still available at this stage.

The outbound report is transmitted by the same mechanism as the post authentication report, including the signature over timestamp and payload and the switch between the asynchronous and the synchronous mode, but to a separate endpoint. The backend stores it in its own index with the request identifier as document identifier, and not as part of the post authentication report. The separation is necessary because the backend treats a second submission with the same identifier as a resubmission of the request and increments a counter that the replay rule evaluates. During development, registering the action of the post authentication interceptor at both hooks produced reports with a submission count of two and would have triggered that rule for every login.

The audit format of the deployment was extended by three tokens, the identifier of the response, the identifier of the request that it answers and the issue time of the response, so that the response level fields can be compared with the audit entry. The Logstash pipeline already mapped the corresponding tokens to fields.

The information obtained from these contexts is extracted, parsed, and combined into a single payload. Before this payload leaves the IdP, the sensitive identifiers it contains are pseudonymised, as required by the design described in Section 4.4.

The implementation of this step is complicated by the fact that it takes place in two different components. The interceptor pseudonymises the identifiers it collects within the Java application, whereas the audit fields are pseudonymised by Logstash during preprocessing. Both must produce the same value for the same identifier, since the correlation described in Section 4.5.1 relies on comparing hashed subjects and addresses across the two sources. A mismatch would not cause an error but would silently prevent records from being matched. Both components therefore apply the same hash function and share the same secret salt. The salt is provided through the deployment configuration rather than embedded in either component, so that it can be rotated without modifying the interceptor or the Logstash pipeline.

5.2 Transport Layer

Before the aggregated data is transmitted, the collecting component authenticates itself to the monitoring tool using an HMAC signature, as outlined in Section 4.4. Both Logstash and the interceptor hold the shared secret and sign the records they send, so that the backend can verify that a received message originates from an authorised collection component.

The verification is conditional on configuration. If no secret is configured on the sending side, the message is transmitted unsigned, and the backend enforces validation only where

a secret has been configured there as well. This makes it possible to deploy the system without prior key distribution, for instance in a test environment, at the cost of accepting unauthenticated records in that case. A production deployment is therefore expected to configure the secret on both sides.

The monitoring tool provides a separate endpoint for each type of collected data. A post authentication report, an outbound report, an audit entry and a configuration document differ in structure and in the processing they require, so that separating them at the interface avoids having to determine the type of a record before it can be handled.

As mentioned in Section 4.4, the interceptors send their payload asynchronously. Neither of them waits for a response from the monitoring tool, and each proceeds with the authentication flow immediately after dispatching its record. If a transmission fails, whether because the backend is unreachable, the request times out or the record is rejected, the interceptor records a warning in the log of the IdP but does not send the record again, so that it is missing in the monitoring tool.

To quantify the cost of this design decision, the interceptors also support a synchronous mode. In this mode each of them waits for the backend to confirm receipt before the authentication flow continues, which makes the time spent on transmission visible in the overall duration of a login. The mode is not intended for productive use but serves as a reference point against which the asynchronous variant can be compared. The resulting measurements are presented in Section 6.4.

The synchronous mode applies to both interceptors and is selected by the same environment variable, so that a login in that mode contains two blocking transmissions. Where a secret is configured, both interceptor endpoints accept only requests that carry a valid signature.

5.3 Correlation and Detection Layer

This section describes the main technical components of the backend application, following the same structure as Section 4.5.

5.3.1 Storage

Elasticsearch serves as the primary data store of the backend application. The choice follows from the data collection layer described in Section 5.1, since Filebeat and Logstash write directly into Elasticsearch and no further conversion is required for the data they deliver.

The application uses separate indices for the different types of data it handles, namely audit entries, metadata, configuration data and the reports of the two interceptors. These indices differ in how they are created. Audit entries are written by Logstash into time based indices that are rotated daily, whereas post authentication reports and outbound reports are stored in a single index each, maintained by the application itself. Queries that span audit data therefore address an index pattern rather than a concrete index, which restricts the set of Elasticsearch operations available for them.

Elasticsearch does not provide relations between documents. The result of a correlation is therefore stored as a field on the post authentication report, which holds the identifier of the audit entry it was matched with. The strategy that produced the match is stored alongside it, as described in Section 4.5.1, so that the basis of a correlation remains visible after the fact.

5.3.2 Correlation

This section describes how the two challenges of the correlation layer identified in Section 4.5.1 are addressed in the implementation, namely the temporal offset between the arrival of related records and the varying availability of fields in the audit log. The post authentication report is static and always contains the fields required for correlation. The constraint therefore lies entirely on the audit side. The audit logging documentation lists all fields that a deployment may record [**AuditLoggingConfiguration**], of which four are usable for establishing a relationship between an audit entry and a post authentication report.

- **I**: the inbound message identifier of the SAML request
- **u**: the username, pseudonymised by Logstash
- **a**: the client IP address, pseudonymised by Logstash
- **T**: the timestamp of the event

The audit format provides further identifiers that are unique per transaction. The `InResponseTo` value of the response echoes the identifier of the inbound request, and a strategy built on it would establish the same one-to-one relationship as the inbound message identifier already does, with no difference in outcome. The assertion identifier and the outbound message identifier are unique as well, but they are created after the post authentication hook has run, which executes before the outgoing message content is constructed [**ProfileHandling**], and are extracted at later points of the audit pipeline [**AuditLoggingConfiguration**]. They are therefore not available to the interceptor at the post authentication hook. The outbound interceptor described in Section 5.1 reads the outbound message identifier and, for assertions that are not encrypted, the assertion identifier, and the audit format of the deployment records the outbound message identifier as well, so that these fields serve the comparison of the two records rather than the correlation. Which identifiers can be used for correlation is thus determined by the observation point rather than by the audit format alone. The remaining fields, among them the entity identifier of the SP, the authentication flow and the released attributes, are identical across many logins of the same partner and narrow the candidate set no further than the SP already does.

These fields differ in how precisely they identify a counterpart. The inbound message identifier is unique per request and therefore establishes the relationship unambiguously, since the same value appears in both records. The two pseudonymised identifiers narrow the set of candidates to those belonging to the same subject and origin, but several logins of the same user may share them. The timestamp alone can only indicate temporal proximity. In addition, all strategies restrict candidates to the SP recorded in both records, since an

audit entry for a different partner can never be the counterpart of a given report. Where the inbound message identifier is not recorded, the relationship can therefore only be approximated. If none of the identifiers is available, the timestamp remains the sole criterion.

The three correlation strategies implement a common interface and are applied in a fixed order of precedence. Each strategy declares the audit fields it depends on and queries the field registry described in Section 5.1 before it is applied. Where a required field is not recorded by the deployment, the strategy withdraws and the next one in the chain is attempted.

The first strategy, `INBOUND_MESSAGE_ID`, matches the inbound message identifier of the post authentication report against the corresponding audit field. Since this value is unique per request, a match establishes the relationship without ambiguity. The second strategy, `HASH_AND_TIME_WINDOW`, narrows the candidates by SP, pseudonymised subject and pseudonymised client address, and additionally restricts them to a configurable time window around the event. The third strategy, `TIME_WINDOW`, retains only the SP and the time window, and is therefore used where neither the message identifier nor the pseudonymised identifiers are recorded.

The pseudonymised subject appears under two names, since the post authentication report stores it as *subjectHash* whereas the audit entry records it as *usernameHash*. Both hold the same value, derived from the same identifier with the same salt, and the distinction is one of naming within the respective data source rather than of content.

Both time window strategies may return more than one candidate. In this case the entry with the smallest temporal distance to the post authentication report is selected. The strategy that produced a match is stored alongside the result, so that the reliability of a given correlation remains apparent afterwards.

The timestamp used for these windows requires particular attention. The field that Elasticsearch assigns to an audit document by default reflects the point at which Filebeat read the log line, not the point at which the authentication took place. Since Filebeat processes log lines in batches, this value is shared by all entries of a batch and does not preserve their order. The Logstash pipeline therefore derives a dedicated field from the event timestamp recorded by the IdP, and both time window strategies operate on this field. The effect of this distinction on correlation precision is reported in Section 6.5.

The temporal offset described in Section 4.5.1 is addressed by repeated attempts. A post authentication report that could not be correlated on arrival is retried periodically, with a minimum age before the first attempt to account for the indexing delay of the log shipping stack. After a defined retry window has elapsed without a match, a separate scheduled sweep marks the report as permanently uncorrelated. This marking is what allows an uncorrelated report to be distinguished from one that is still awaiting its counterpart, and thus what makes the proportion of successfully correlated reports determinable at all.

Two mechanisms prevent an audit entry from being claimed twice. Within a single retry run, entries that have already been assigned are excluded from the candidate set of subsequent reports. Across runs, the application verifies against the stored data whether an audit entry has already been referenced by another report before accepting a match.

5.3.3 Detection

The detection logic is divided into four engines, following the separation by input type described in Section 4.5.2. The rule engine evaluates login events derived from audit entries, the configuration rule engine evaluates the security settings of an SP, the metadata rule engine evaluates its metadata, and the interceptor rule engine evaluates the post authentication reports produced by the interceptor.

All rules implement a common interface. A rule receives a single object of the type its engine is responsible for and returns either an alert or nothing. Each rule additionally declares which fields it depends on, and the engine queries the field registry described in Section 5.1 before evaluating it. Where a required field is not recorded by the deployment, the rule is skipped rather than evaluated on incomplete data.

The engines are executed by a scheduler at intervals that reflect the rhythm of their input. Login events and post authentication reports arrive continuously and are therefore evaluated frequently, whereas configuration and metadata change rarely and are checked at longer intervals. To avoid evaluating the same login events repeatedly, the application maintains a marker that records the point up to which audit entries have already been processed. Each run evaluates only entries newer than this marker, and the marker is advanced once the run has completed. A grace period is applied before an entry is considered, so that records still being indexed by the log shipping stack are not skipped.

A further mechanism concerns repeated findings. A weakness in a configuration persists until the operator changes it, and evaluating the same configuration every few minutes would otherwise produce a continuous stream of identical alerts. The engines for the configuration, the metadata and the post authentication reports therefore store at most one alert of a given type per SP and day. The engine for login events stores at most one alert of a given type per audit entry, so that a rule that examines a property of every login, such as an unencrypted assertion, raises an alert for every affected login. Rules that aggregate several logins limit themselves further, `BRUTE_FORCE` to one alert per subject and day and `ASSERTION_REPLAY` to one per assertion and day.

The rules differ in the kind of condition they examine. Three examples illustrate this range. `WEAK_ENCRYPTION` compares the encryption algorithm recorded for a login against a list of algorithms considered insecure, which is maintained in the application and can be adjusted without changing the rule itself. `BRUTE_FORCE` does not evaluate a single event in isolation but counts failed authentication attempts of the same subject within a defined time window, which requires a query across previously stored events. `SIGNATURE_WRAPPING_SUSPECTED` operates on the raw SAML request contained in a post authentication report and examines its structure for duplicate identifiers and for signature elements referencing an identifier that occurs more than once.

The remaining rules follow one of these patterns or compare an observed event against the configuration declared for the SP involved, as described for `MFA_BYPASS` in Section 4.5.2. Their assignment to documented vulnerabilities and attack classes is given in Table 4.1.

A further difficulty arose with the comparison between configuration and runtime event. The authentication flows permitted for an SP are listed in *relying-party.xml* without

a namespace prefix, whereas the audit log records the flow that was actually used with one. The same flow therefore appears as `MFA` in the configuration and as `authn/MFA` in the audit entry. A direct string comparison between the two never matches, and the rule would consequently never report a finding. The comparison is therefore performed token by token and accepts both notations. It is implemented in a shared comparison function that is used by all rules evaluating authentication flows. As with the two cases described in Section 5.1, the failure mode here would have been silent, since neither an exception nor a log entry would have indicated that the comparison was ineffective.

Two rules operate on pairs of records and are therefore not evaluated by the rule engines. Both are triggered by the correlation service. When a report is marked as permanently uncorrelated, either because the retry loop reaches the end of its window or because the periodic sweep finds it expired, the service raises a `CORRELATION_DISCREPANCY` alert of medium severity. Each report is expired exactly once, so that this alert cannot produce repeated findings.

When a report is correlated with an audit entry, the service loads the outbound report with the same request identifier and passes it together with the audit entry to the consistency service. If no outbound report exists, the check is skipped. Otherwise the service compares the fields that both records contain, namely the status, the identifier of the answered request, the identifier and the issue time of the outbound message and the encryption state, and for an unencrypted assertion also the identifier of the assertion, the authentication instant, the authentication context, the format of the name identifier and the pseudonymised name identifier. A field is compared only if the interceptor observed a value for it and the field registry shows that the audit format of the deployment records it. If the audit entry lacks a value for a field that the registry lists, the field counts as deviating, since a removed field is itself a sign of tampering. Timestamps are parsed and compared as instants, and the other values as strings after the abbreviations of the audit format have been mapped back to the notation of the report. The map mirrors the default map of the IdP and is a copy that is not synchronised with its configuration. On a deviation the service stores a `FIELD_MISMATCH` alert of high severity, whose details list each deviating field with the values of both records.

5.4 Visualisation Layer

The visualisation layer displays the collected, correlated, and detected data through an Angular based web application. A generic dashboard such as Kibana was not used, since the views required here are specific to the domain of the monitoring tool. They present findings together with the records from which they were derived and follow the relationships that the correlation layer establishes, neither of which can be expressed through generic index queries alone. The main pages implemented for the web application are described below.

The dashboard provides an overview of the most important data within a specified timeframe. It includes the total number of logins, failed logins, successful logins, and detected alerts. In addition, it provides a line chart showing the number of logins over the

selected timeframe, an overview of the total number of logins per SP, and a detailed table of the individual login events.

The login overview for each SP also provides a detailed dialog containing the configuration and metadata defined for the respective SP in the Shibboleth IdP. The SP details further include an overview of any misconfigurations detected by the rule engines described in Section 5.3.3.

The second major page of the web application is the alert page. As its name suggests, this page displays the recent alerts detected by the monitoring tool. It includes a vertical bar chart showing the alerts detected during the current week, grouped by their respective severity. The page also provides a table of alerts ordered by severity.

Each row in the alert table is interactive and opens a detailed dialog containing information about the selected alert. This dialog also provides access to the audit entry and the post authentication report associated with the alert. Conversely, both the audit entry and the post authentication report dialogs list all alerts detected for the corresponding event. The relationship established by the correlation layer is therefore navigable in both directions, which allows an operator to examine a finding together with the records it was derived from rather than in isolation.

The audit entry view displays only the information that was available in the corresponding audit file of the Shibboleth IdP. Since subjects and client addresses are pseudonymised before leaving the IdP, as described in Section 5.1, the interface presents these values in their hashed form. An operator can therefore recognise that several events originate from the same subject without learning the identity of that subject.

Each alert can also be acknowledged by an operator and marked as a false positive. Acknowledged alerts and false positives are removed from the default alert view, which only displays active alerts.

The third major page displays the collected post authentication reports in a table. Each row contains the most relevant information from the corresponding post authentication report and indicates whether the report was successfully correlated with an audit entry. This makes the state described in Section 5.3.2 visible to the operator, including reports for which no counterpart was found within the retry window. Each row can be expanded to reveal additional details, including the raw SAML request, and a button allows the operator to open the corresponding alert in a detailed dialog.

6 Evaluation

This chapter evaluates the proposed monitoring system. It presents the test setup, evaluates the correlation and detection capabilities, measures the performance overhead introduced by the interceptors, lists the defects that the evaluation uncovered, and concludes with a discussion of the threats to validity.

6.1 Test Setup

To facilitate integration with the Java testing framework, the test environment runs in containers. This comprises the Shibboleth IdP, an LDAP directory serving as the user store, the SPs and the log shipping stack. Only the backend application of the monitoring tool and the test code run directly on the host.

Two SPs are deployed, referred to as the weak SP and the clean SP in the following. The weak SP is used for the detection scenarios and is configured to exhibit the weaknesses that the rules are intended to identify. The clean SP encrypts its assertions and serves as a reference for the false positive rate, since any alert raised for a legitimate login against it is by definition a false positive.

The evaluation distinguishes three levels of testing. Unit tests examine individual components in isolation, with their dependencies replaced by mocks, so that a test result reflects the behaviour of that component alone. Integration tests run against a real Elasticsearch instance, which additionally covers the persistence layer, the index mappings and the queries that a mock cannot represent. End-to-end tests exercise the complete pipeline, from a login at the SP through the log shipping stack to the alert produced by the backend.

Logins are generated in two ways. An HTTP client harness performs the SAML login sequence directly, parsing the forms returned by the IdP and submitting them without rendering the pages. Selenium based tests drive an actual browser through the same sequence. The two differ in the interval between consecutive logins, since the harness proceeds as fast as the server responds whereas a browser adds the time required for page loading and form interaction. This difference is relevant for the correlation measurements in Section 6.2, where the temporal density of logins determines how reliably records can be matched.

The environment is not a production deployment. It comprises a single IdP, two SPs and a small number of test accounts, and all traffic is generated for the purpose of measurement. The implications of this are discussed in Section 6.6.

6.2 Correlation

Section 5.3.2 described the correlation mechanism, which links each post authentication report to the audit entry of the same login. It first tries `INBOUND_MESSAGE_ID` and falls back to two heuristic strategies where the identifier is not available. Both heuristics query

the audit entries of the same SP within a time window around the report. `HASH_AND_TIME_WINDOW` additionally requires the same *usernameHash* and *ipHash*, whereas `TIME_WINDOW` relies on the time alone. The entries returned by such a query are referred to as candidates in the following, and the strategy selects the candidate with the smallest temporal distance to the report. This section examines how often the selected candidate is the correct one (precision) and how many reports find a counterpart at all (coverage). The exact strategy serves as the reference for the heuristics, and the measurement is repeated on datasets that differ in the number of users and in the login generator.

6.2.1 Method and Datasets

Ground truth. To determine the precision of the heuristic strategies, a reference is required that indicates whether a given correlation was correct. The correlation via `INBOUND_MESSAGE_ID` establishes a one-to-one relationship, since the identifier is unique per request. Wherever this strategy produced a match, the correct counterpart is therefore known without ambiguity, which allows it to serve as a ground truth for the heuristic strategies. An analysis program in the test sources of the backend proceeds as follows:

- It collects all post authentication reports of a configurable time period that were correlated via `INBOUND_MESSAGE_ID`.
- For each report and each window size from ±1 to ±60 s, it executes the query of the heuristic strategy unchanged and selects the same candidate as the productive system would, namely the one with the smallest temporal distance.
- It compares the selected candidate with the known correct audit entry and classifies the outcome as a hit, an incorrect assignment or no match. Precision is the share of hits among the assigned reports, coverage the share of reports that were assigned at all.
- It records the offset between the reception of the report and the event time of its audit entry, and the distance to the nearest other audit entry of the same SP as a measure of the local login density.

In operation, an audit entry that has already been assigned to a report is no longer offered as a candidate for other reports. The analysis has to omit this filter, since every audit entry in the ground truth has already been assigned to its correct report, so that the filter would remove exactly the candidate that is being looked for. Each report is therefore evaluated against all audit entries in its window.

Datasets. The heuristics only take effect when the audit log lacks the inbound message identifier, for instance because an operator does not record it or uses an older audit format. Whether they then select the correct entry depends on two properties of the traffic: how many logins at the same SP fall into the same time window, and whether these logins can be told apart by subject and address. Both vary considerably in operation. At the start of a lecture or during an exam registration, many users log in at the same SP within a few seconds, while at night a single login may stand alone. Some users log in from an

address of their own, while others share one behind a gateway. The five datasets reproduce these situations step by step, each adding a condition that the previous ones leave open. Table 6.1 summarises how they were generated. Coverage is additionally determined on the performance series of Section 6.4, which comprises 7,236 reports and is by far the largest dataset.

The starting point is the simplest case, a single user who logs in repeatedly at the same SP (D1). For the heuristics, this case removes every distinguishing feature except time. All candidates share subject, address and SP, so that the hashes exclude nothing and both heuristics behave like `TIME_WINDOW`. The correct entry can only be found if the time recorded for each login is precise enough to keep consecutive logins apart. D1 therefore tests the time base on which both heuristics rest. If it failed here, none of the further scenarios could succeed.

In operation, however, the logins within a window rarely stem from one person. In D2, three users log in at the same SP at the same time, so that candidates of different users lie in the same window. This is precisely the situation for which the hashes exist. `HASH_AND_TIME_WINDOW` can discard the entries of other users, whereas `TIME_WINDOW` has to choose among all of them by time alone. D2 is thus the first dataset in which the two heuristics differ, and it shows what an operator loses who records neither the subject nor the address in the audit log.

D1 and D2 are generated by the HTTP harness, which starts the next login as soon as the server has answered. It produces logins at intervals that no user reaches with a browser, since loading pages and filling in forms take time. If the harness is considerably denser than real traffic, D2 overstates how often the time alone fails. D3 therefore repeats D1 with Chrome driven by Selenium, which loads every page and submits the forms as a browser does, and shows how closely spaced the logins of one user are under real browser timing.

D3 still covers only one user. D4 combines the two previous steps: three users log in simultaneously, each through its own browser, which comes closest to several people logging in at the same time. It shows whether the weakness of `TIME_WINDOW` that D2 exposes also occurs under browser timing, and how often. Since every browser runs in its own container, each user also has an address of their own, as users at different devices or in different networks do. This makes it possible to analyse the same data a second time without *usernameHash* and to determine whether *ipHash* alone keeps the users apart.

Separate addresses cannot be taken for granted, however. Behind a gateway with network address translation, for instance in a company network, a dormitory or a public wireless network, many users appear under the same address, and *ipHash* no longer distinguishes them. The separation then rests on *usernameHash* alone. D5 reproduces this case by running three users as threads of a single process, so that all logins originate from one address.

6.2.2 Results

Table 6.2 summarises the precision of both heuristics in the five datasets. The figures are identical for all window sizes from ±1 to ±60 s, for a reason explained below. The

Table 6.1: *Datasets of the correlation measurements*

Dataset	Login generator	Users	Logins per user	Reports
D1	HTTP harness	1	200 + 3 warm-up	203
D2	HTTP harness	3	200	600
D3	Selenium	1	200 + 2 warm-up	202
D4	Selenium, one container per user	3	200 + 2 warm-up	606
D5	HTTP harness, threads of one process	3	200	600

following paragraphs state what the figures show, from the coverage of the correlation to the limits of the heuristics.

Table 6.2: *Precision of the heuristic strategies per dataset*

Dataset	Scenario	`HASH_AND_TIME_WINDOW`	`TIME_WINDOW`
D1	Single user	100%	100%
D2	Several users	100%	65.0%
D3	Real browser	100%	100%
D4	Real browser, several users	100%	89.9%
D5	Shared address	100%	73.7%

Every report finds its counterpart. In the performance series, all 7,236 post authentication reports were correlated with their audit entry through the inbound message identifier, and none expired without a counterpart. The coverage of the exact strategy was therefore 100 percent, with a lower bound of 99.95 percent for the 95 percent Wilson interval [**Wilson**]. That this figure can be determined at all is due to the sweep described in Section 5.3.2, which marks reports without a counterpart as permanently uncorrelated after a waiting period, so that a lost report can be told apart from one that is still waiting. The heuristics, too, assigned every report in all five datasets, even with the smallest window of ±1 s. This follows from the offsets in Table 6.4: a report arrives at most about 160 ms before or after the event time of its audit entry, so that the correct entry always lies within the window. Where a heuristic fails, it therefore assigns a report to the wrong counterpart rather than to none, which is why the following paragraphs concern precision.

With the hashes, the heuristic is as reliable as the exact strategy. `HASH_AND_TIME_WINDOW` selected the correct entry for every report in all five datasets. This includes D1, where the hashes exclude nothing and the event time alone decides, and D5, where all users share one address. The size of the window does not affect the precision. In D1 the mean number of candidates rises from 6.4 at ±1 s to 197.6 at ±60 s without a single wrong assignment, since a larger window only adds candidates that lie further away and are therefore never selected. The window therefore only has to be large enough to contain the correct entry, and a larger window costs nothing but query effort. In the test environment ±1 s would have sufficed, but in operation a report can arrive later, since the interceptors

give up a transmission only after 1.5 s, and on separate hosts the clocks may differ. The default of ±10 s covers both with a wide margin, at the price of examining 58.1 instead of 6.4 candidates per report in D1.

Without the hashes, wrong assignments occur only between logins less than 100 ms apart. `TIME_WINDOW` reached 65.0 percent in D2, 73.7 percent in D5 and 89.9 percent in D4. Table 6.3 breaks these figures down by the distance of each login to the nearest other login at the same SP, with the number of logins in each range in parentheses. All wrong assignments concern logins with a distance below 100 ms. At larger distances, `TIME_WINDOW` was correct in every case. Within this zone, it selected the correct entry in 56 to 69 percent of cases, regardless of whether the logins came from the harness or from browsers. The differences between the overall figures therefore do not stem from the strategy but from how many logins fall into the zone: 87 percent in D2, 84 percent in D5 and 23 percent in D4.

Table 6.3: *Precision of `TIME_WINDOW` by distance to the nearest other login at the same SP*

Distance	D2	D3	D4	D5
< 100 ms	59.7% (521)	— (0)	55.8% (138)	68.7% (504)
100–300 ms	100% (65)	— (0)	100% (132)	100% (96)
300 ms–1 s	100% (14)	— (0)	100% (258)	— (0)
> 1 s	— (0)	100% (202)	100% (78)	— (0)
Share below 100 ms	86.8%	0%	22.8%	84.0%

Real browser traffic reaches this zone far less often. With a single browser user (D3), every login lay more than one second away from the next, and a window of ±1 s contained on average one candidate instead of 6.4 with the harness. Even with three users logging in simultaneously through browsers (D4), less than a quarter of the logins fell below 100 ms. The weakness of `TIME_WINDOW` is thus not an artefact of the harness, since it is just as pronounced within the zone, but the harness reaches the zone far more often than browser traffic does. How often a productive IdP reaches it depends on its load at peak times and cannot be derived from the test environment.

Each hash covers the case in which the other one is uninformative. In D4, every user had an address of their own. Analysed without *usernameHash*, the heuristic found exactly the same candidates as with both hashes, for every window size and including the 138 logins below 100 ms, and thus again reached 100 percent. *ipHash* alone therefore separates users who log in from different devices or networks. In D5, where all users shared one address, the same variant produced exactly the figures of `TIME_WINDOW`, since a filter on the only address present excludes nothing. `HASH_AND_TIME_WINDOW` nevertheless remained at 100 percent, because *usernameHash* separated the users on its own. In all tested scenarios, the heuristic thus stays exact as long as at least one of the two identifiers distinguishes

the users, and it falls back to the density dependence of TIME_WINDOW only when both are missing or uninformative.

Operating limit of the heuristics. Table 6.4 shows the offset between the reception of a report and the event time of its audit entry. This was not recorded for D2. It is mostly negative, by about 30 ms, because the post authentication interceptor runs in the middle of the profile flow, whereas %T marks the completion of the transaction. Since the heuristics select the entry closest to the reception time, a neighbouring login can only be selected instead of the correct one if the two lie closer together than twice the largest absolute offset (last column). Across all datasets, this critical distance is at most 316 ms (D4), which corresponds to about three logins per second. For HASH_AND_TIME_WINDOW, the limit applies to logins of the same subject from the same address at the same SP, a rate that interactive users do not reach. For TIME_WINDOW, it applies to all logins at an SP, which explains why this strategy fails in dense traffic. The bound is conservative, since wrong assignments were only observed below 100 ms. All offsets were measured on a single host, where the IdP and the backend share one clock. On separate hosts, the difference between the clocks adds to the offset and lowers the limit, so that the offset, which the analysis records, has to be measured again in a different environment.

Table 6.4: *Offset between the reception of a report and the event time of its audit entry*

Dataset	Minimum	Maximum	Critical distance
D1	−98 ms	+75 ms	196 ms
D3	−102 ms	−17 ms	204 ms
D4	−158 ms	+42 ms	316 ms
D5	−50 ms	+110 ms	220 ms

Taken together, the correlation found a counterpart for every report, and with the inbound message identifier or at least one separating hash, it found the correct one in every tested scenario. Only an audit log that records neither the identifier nor the hashes leaves the correlation to time alone, and it then assigns reports to the wrong entry where logins at the same SP follow each other within less than 100 ms. Operators should therefore record %I in the audit format or, where that is not possible, at least the subject and the address.

6.3 Detection

Section 5.3.3 described the detection layer, which evaluates fifteen rules in four engines, one for each type of input: the login events derived from the audit log, the configuration of an SP, its metadata, and the post authentication reports. Most rules examine a single input for a condition that indicates a weakness or an attack, for instance an assertion that was sent unencrypted or a series of failed logins for the same subject. Two rules compare records that were produced independently of each other. The consistency check compares the outbound report with the audit entry of the same login and raises FIELD_MISMATCH

when the two disagree, and CORRELATION_DISCREPANCY reports a post authentication report for which no audit entry arrives within the waiting period.

This section examines whether each rule behaves as specified, that is whether it raises an alert when the condition it examines occurs and remains silent when it does not. Since no productive logins with a known classification are available, the rules are tested in scenarios whose expected outcome is fixed in advance, on the test levels introduced in Section 6.1, and series of legitimate logins show whether alerts arise where none are due.

6.3.1 Method and Scenarios

Proofs. The tests establish two kinds of proof. A true positive proof constructs a scenario in which a rule is supposed to raise an alert and confirms that it does. A true negative proof constructs a scenario in which no alert is due and confirms that none is raised. How much a proof shows depends on the test level at which it is established. In an integration test, the test builds the input of the rule itself, for instance an audit entry or a post authentication report, stores any history the rule requires in Elasticsearch and passes the input directly to the rule engine rather than waiting for the scheduler, so that the result does not depend on its timing. Such a proof covers the rule, its queries and the stored documents, but not the way in which Filebeat and Logstash turn a real audit line into such an input. An end-to-end test starts from a real login in the browser and covers this path as well, which makes it the only level at which a fault in the log shipping stack can become visible.

Scenarios. In operation, the detection layer has to meet two expectations at once. It has to report the weaknesses and attacks that the rules describe, and it has to remain quiet during the vast majority of logins, which are legitimate, since an operator who receives alerts for ordinary logins soon stops reading them. The two comparison rules carry a third expectation, namely to notice when a record has been altered or has gone missing. The scenarios address these expectations in turn and finally repeat the second one under sustained load.

Every rule has at least one scenario at the integration level that contains the condition it is meant to detect. For most rules this is a single prepared input, such as an audit entry that records an unencrypted assertion or a SAML request with two signature elements, a typical trace of a signature wrapping attack, which the rule treats as a possible signature wrapping attempt [**BreakingSAML**]. Rules that depend on history receive it in the same way. For BEHAVIOUR_ANOMALY the test first stores five logins of a user from a known address and then one from a new address, and for CORRELATION_DISCREPANCY it stores a post authentication report that arrived twenty minutes earlier, so that the waiting period of fifteen minutes has already elapsed. Wherever a regular login in the test environment can produce the condition, an end-to-end scenario follows: a login with the user agent of a script, a login with a wrong password, five failed logins in quick succession, which exceeds the threshold of three within five minutes, and a login at the weak SP, whose assertions are not encrypted. For the remaining rules, an end-to-end scenario would require more than a regular login, for instance a second browser with another address, an SP that offers

an MFA flow, a crafted or replayed message, a changed configuration or metadata of an SP, or log shipping that stays interrupted for longer than the waiting period. These scenarios were not implemented in this work. Table 6.5 names the requirement for each rule.

The second expectation is tested with legitimate logins through the browser. At the clean SP, three users log in alternately, 100 times in total, and after a further 100 seconds, in which the scheduler evaluates all entries, none of the logins may have raised an alert. Since every alert in this run would be a false alarm, the run is at the same time a true negative proof for every rule that can react to a login. The same run against the weak SP tests the consistency check in the presence of the alerts that the weaknesses of this SP rightly cause, so that there no `FIELD_MISMATCH` alert may arise. In addition, a single legitimate login at the clean SP provides the end-to-end true negative proofs of the five rules marked TN in Table 6.5.

The third expectation concerns someone who alters the audit entry of a login after the fact, for instance to conceal which request a response answered. The consistency check is meant to notice this, since the outbound report is produced independently by the IdP and reaches the backend directly. It compares the fields that both records contain and that the audit format records. For every login these are the status, the identifier of the request answered by the response, the identifier and the issue time of the outbound message and the encryption state. For an unencrypted assertion, the identifier of the assertion, the authentication instant, the authentication context, the format of the name identifier and the pseudonymised name identifier are compared as well. To produce a manipulation, a login against the clean SP was performed while the backend was stopped. After the audit entry had reached Elasticsearch, the identifier of the answered request was replaced by a different value, and the backend was started again.

Finally, the comparison rules must not raise false alarms when many logins arrive in quick succession, since both rely on the records of every login being matched in time. The performance series of Section 6.4 provides this load with 10,854 logins against the weak SP, in measurements of about 200 logins each. In 7,236 of them the interceptors were active and transmitted both reports. The remaining logins belong to the baseline configuration without interceptors.

6.3.2 Results

Table 6.5 shows which rules are also tested end-to-end and what an end-to-end test of the others would require. The following paragraphs state what the tests establish, from the coverage of the rules to the limits of what the results can show.

Every rule is proven against a real Elasticsearch instance, and six are also tested end-to-end. All fifteen rules have a true positive proof on the unit and on the integration level, so that for every rule the queries, the index mapping and the stored documents are covered and not only the detection logic. The test suite comprises 118 unit tests for the rules, 21 integration tests and seven end-to-end tests. Six rules are additionally proven from a real login. For `SUSPICIOUS_USER_AGENT`, `FAILED_LOGIN`, `BRUTE_FORCE` and `UNENCRYPTED_ASSERTION`, the end-to-end level contains both a true positive and a true

Table 6.5: *End-to-end coverage per rule*

Rule	End-to-end	Note
SUSPICIOUS_USER_AGENT	TP, TN	—
FAILED_LOGIN	TP, TN	—
BRUTE_FORCE	TP, TN	—
UNENCRYPTED_ASSERTION	TP, TN	—
WEAK_ENCRYPTION	TN	TP requires an SP with a weak algorithm
FIELD_MISMATCH	TN, TP (manual)	—
BEHAVIOUR_ANOMALY	—	Requires a second browser with another address
MFA_BYPASS	—	Requires an SP that offers an MFA flow
ASSERTION_REPLAY	—	A replayed assertion is presented to the SP, not to the IdP
SIGNATURE_WRAPPING_SUSPECTED	—	Requires a crafted request that the IdP accepts
REQUEST_REPLAY	—	Requires a replay after the replay cache of the IdP has expired
WEAK_CONFIGURATION	—	Requires a changed SP configuration
MISSING_MFA	—	Requires a changed SP configuration
CERTIFICATE_EXPIRED	—	Requires SP metadata with an expired certificate
CORRELATION_DISCREPANCY	—	Requires log shipping interrupted for 15 minutes

negative proof. WEAK_ENCRYPTION has only the true negative proof, since no SP in the test environment requests a weak algorithm, and the true positive proof of FIELD_MISMATCH is the manipulation described below. For the remaining nine rules, the last column of the table names what an end-to-end scenario would require. That the end-to-end level is not redundant became apparent when it uncovered a fault in the log shipping stack that no integration test could have found (Section 6.5).

Legitimate logins raise no false alarms. In the 100 logins at the clean SP, none of the logins raised an alert, and in the 100 logins at the weak SP, where the alerts caused by its weaknesses are expected, no FIELD_MISMATCH was raised. In all 200 logins, the post authentication report was correlated with its audit entry through the inbound message identifier, and the consistency check passed, with five compared fields at the clean SP and ten at the weak SP (Table 6.6). No false alarm in 100 logins does not prove that false alarms cannot occur, but it makes a rate above 3.7 percent per SP unlikely, with 95 percent confidence according to the Wilson interval [**Wilson**]. For both SPs together, the limit is 1.9 percent. At the clean SP this applies to every alert that a login can trigger, at the weak SP to the consistency check. Findings about the configuration of an SP are not triggered by logins. The clean SP, for instance, receives a MISSING_MFA alert once per day, since it does not offer an MFA flow. Sustained load did not change this. During the 10,854 logins of the performance series, the detection layer raised 21,712 alerts, all

of them expected: one UNENCRYPTED_ASSERTION and one SUSPICIOUS_USER_AGENT alert per login, since the benchmark addresses the weak SP and is not a browser, and four configuration alerts. Since these two rules report every affected login rather than the condition once, a persistent weakness of an SP produces one alert per login. None of the alerts was a FIELD_MISMATCH or CORRELATION_DISCREPANCY alert, although the 7,236 logins with active interceptors passed through the consistency check. These figures apply to the two SP configurations and to the login flow of the test environment. Legitimate but unusual configurations were not tested.

Table 6.6: *Consistency check for legitimate logins*

SP	Logins	Fields compared	Mismatches	Upper bound (95 %)
Clean SP	100	5	0	3.7 %
Weak SP	100	10	0	3.7 %

The comparison rules detect an altered or a missing record. In the manipulation scenario, the backend matched the post authentication report to the altered audit entry through the unchanged inbound message identifier within two seconds of its start. The consistency check then raised a FIELD_MISMATCH alert of high severity that named exactly the altered field and no other. The check thus detects a contradiction between two independently produced records of the same login. Since only one field of a single entry was altered, the test shows that the mechanism works, not how reliably it detects manipulations in general. A manipulation can only become visible in the fields that the check compares, and the check establishes that the two records disagree, not which of them was altered. For CORRELATION_DISCREPANCY, the integration test confirms that a report without a counterpart is marked as expired after the waiting period and raises the alert. An audit entry that arrives only afterwards, for instance because of a delay in the log shipping stack, is no longer correlated, so that the alert remains although the record exists.

The results establish the fidelity of the implementation, not its detection quality in operation. The rules were derived from documented weaknesses and requirements (Section 3.4), and the scenarios reproduce the conditions that the rules describe. The results therefore show that the implementation behaves as specified: every rule raises its alert when its condition occurs, and legitimate logins remain without false alarms. How often the rules detect real attacks in operation, and how many of their alerts would be false for productive traffic with its unusual but legitimate configurations, cannot be derived from them. That would require productive logins labelled as legitimate or malicious independently of the rules, which no test environment can produce.

6.4 Performance Overhead

The two interceptors run inside the IdP, the post authentication interceptor as a *post AuthenticationFlow* and the outbound interceptor as an *outboundInterceptorFlow*. Both therefore lie on the critical path of every login, so that any time they take is added to the time a user waits for access to the SP, and any processor time they use is taken from the IdP. The evaluation addresses two questions. The first is how much the interceptors add to a login, in latency and in processor time, in the configuration intended for operation. The second concerns the way in which they transmit their records. As described in Section 5.2, they send them without waiting for a response, so that a login does not depend on the backend. A failed transmission is only recorded in the log of the IdP, and the record is not sent again. Waiting for a confirmation would allow the login to react to such a failure, and the second question is what waiting would cost and whether the choice of fire-and-forget transmission is justified.

Three configurations are compared. In *baseline*, neither interceptor is configured for the SP, so that a login runs as in an IdP without monitoring. In *async*, both interceptors are active and transmit their records without waiting, which is the configuration intended for operation. In *sync*, both are active but wait for the confirmation of the backend, so that every login contains two blocking transmissions. The comparison of *baseline* and *async* answers the first question, and the comparison of *async* and *sync* the second. Both are measured as the client experiences a login and as processor load of the IdP, in repeated runs that compare all three configurations under the same conditions.

6.4.1 Method and Measurements

Measurements. Three measurements capture the cost of the interceptors from different angles. The internal measurement is recorded by each interceptor itself (*executionTimeMs*) and covers the collection of the data, the hashing and the extraction of fields. It excludes the construction of the payload and the transmission and is therefore a lower bound. Since the value is part of every report, it is available without further instrumentation. The external measurement is the duration of a login as the client observes it, from the first request to the point at which the SP grants access. Since it includes every step of the login, it shows the delay that a user actually experiences. The third measurement records the processor utilisation of the IdP container about every two seconds through `docker stats`, where 100 percent corresponds to one fully used logical processor. Only the samples taken during the measured logins are evaluated, so that the warm-up, in which class loading and compilation in the freshly started IdP raise the utilisation, is excluded. This measurement captures a cost that appears as additional computation rather than as additional waiting time, which the two latency measurements could miss. Since the configurations do not complete the same number of logins per second, the processor time per login is derived by dividing the mean utilisation by the throughput. The mean is used here because, unlike the median, it adds up to the total processor time of a measurement.

Procedure. All logins are generated by the HTTP harness for a single test account against the weak SP. This is the less favourable case for the interceptors, since the outbound interceptor evaluates all fields of the unencrypted assertion and a login of the baseline is cheaper without encryption, so that the relative overhead turns out larger rather than smaller. Each login uses a fresh client with an empty cookie store, so that it passes through the complete authentication without reusing a session, and the next login starts as soon as the previous one has completed. The series comprises 18 runs, each of which measures all three configurations, which yields 54 measurements. For every measurement, the IdP container is recreated with the settings of the configuration, and a single check login verifies that the configuration is in effect by testing whether the expected reports arrive at the backend. One hundred warm-up logins follow, whose values are discarded, and then the one hundred logins that are measured. The backend runs during all measurements, including the baseline, so that the environment is the same, and a pause of four minutes after each measurement gives it time to process the records. The whole series took about five hours and comprised 10,854 logins.

Within a session, later measurements tended to be slower than earlier ones. In this series, the third position of a run was on average about 10 ms slower than the first, which is of the same order as the effects to be measured, although the difference is not statistically established. Plausible causes are the memory use of the host, with about 12 of the 15 GiB in use during the series, and the frequency scaling of the notebook processor. Neither was examined further. As a precaution against attributing such a drift to a configuration, the order of the configurations is rotated between runs, so that each configuration occupies each of the three positions six times.

Analysis. The run, not the individual login, is the unit of the analysis. The hundred logins of a measurement share the same JVM, the same container and the same load, so that they are not independent observations. The medians of different runs differ considerably more than the scatter of the logins within a run would suggest, so that treating the logins as independent would overstate the precision. Each measurement therefore contributes the median of its hundred logins and its processor time per login, which yields 18 values per configuration. The slower logins are described by the 90th percentile of each measurement. The differences between the configurations are estimated with a linear model that accounts for the run and the position within the run besides the configuration, and they are reported with their 95 percent confidence interval. As a simple check, the number of runs in which one configuration was slower than the other is counted as well. With 18 runs, the confidence intervals of the latency differences have a half width of about 11 ms, so that smaller differences cannot be distinguished from zero.

6.4.2 Results

Table 6.7 summarises what a login costs in each configuration, as the mean over the 18 runs. The following paragraphs explain the figures, from the work inside the interceptors to the answers to the two questions.

Table 6.7: *Cost of a login per configuration*

	baseline	async	sync
Median login	356.5 ms	362.7 ms	416.1 ms
Difference to baseline	—	+6.2 ms	+59.6 ms
95 % confidence interval	—	[−4.7; +17.1] ms	[+48.7; +70.4] ms
Difference at the 90th percentile	—	+8.1 ms	+92.0 ms
Logins per second	2.77	2.71	2.36
Processor time per login	126 ms	154 ms	166 ms

The work inside the interceptors takes less than half a millisecond. The internal measurement yields a median of 0.415 ms for the post authentication interceptor and 0.039 ms for the outbound interceptor, and 99 percent of the reports stay below 2 ms. Together this is about 0.45 ms, or 0.13 percent of a median login in the baseline. Since the measurement excludes the construction of the payload and the transmission, it is a lower bound, but it shows that collecting and pseudonymising the data does not weigh on the login. Any larger difference between the configurations must therefore stem from building and sending the reports.

Asynchronous transmission adds no latency that the experiment can distinguish from zero. A median login takes 6.2 ms longer than in the baseline, and the 95 percent confidence interval ranges from −4.7 to +17.1 ms and thus includes zero. In twelve of the 18 runs the asynchronous configuration was the slower one, which is a weak tendency but not an established effect. The slower logins behave in the same way, with a difference of 8.1 ms at the 90th percentile and an upper bound of 22 ms, or 5.2 percent. With 95 percent confidence, the overhead therefore lies below 17.1 ms, or 4.8 percent of a median login. Smaller effects, such as the point estimate suggests, are below the resolution of the experiment.

Synchronous transmission adds about 60 milliseconds to every login. A median login takes 59.6 ms or 16.7 percent longer than in the baseline, with a 95 percent confidence interval from +48.7 to +70.4 ms, and the synchronous configuration was the slower one in all 18 runs. Compared with the asynchronous configuration, the difference is 53.4 ms. Since every login contains two blocking transmissions, each of them costs roughly 27 to 30 ms. The slower logins are affected more strongly, as the difference grows to 92.0 ms or 21.6 percent at the 90th percentile. Since both configurations build and send the same reports, this difference is the time spent waiting, during which the backend validates each report and stores it in Elasticsearch before it answers.

Asynchronous transmission costs processor time, and waiting does not save it. In the asynchronous configuration, the IdP spends 28 ms more processor time on a login than in the baseline, with a 95 percent confidence interval from 19 to 37 ms. This is 22 percent of the 126 ms that a login costs without interceptors. The synchronous configuration shows a lower utilisation than the asynchronous one because it completes fewer logins per

second, 2.36 instead of 2.71, since each login waits for the backend. Per login, however, it does not use less processor time than the asynchronous configuration. The additional processor time does not show in the latency of the asynchronous configuration because the transmission runs in background threads. At the load of the test it occupies less than half a percent of the 16 logical processors of the host, but it grows with the number of logins, so that 50 logins per second at a peak would require about 1.4 additional logical processors.

In the configuration intended for operation, the interceptors therefore add no measurable latency and about 28 ms of processor time to a login, which answers the first question. Waiting for a confirmation would add about 60 ms to every login and more to the slow ones, without saving processor time. In the current implementation it would not even improve the delivery, since both modes only log a failed transmission and neither sends the record again. A delivery guarantee would require a local buffer that repeats failed transmissions, and such a buffer works with asynchronous transmission as well. The measurement therefore supports the choice of fire-and-forget transmission described in Section 5.2.

6.5 Defects Found

Besides the results reported above, the evaluation uncovered six defects that operation itself had not revealed. In each case the system kept running and produced plausible output, and the defect only became visible when the behaviour was compared with an expectation defined in advance. Table 6.8 lists them together with the consequence they would have had in operation. After the missing identifier had been added to the audit format, the exact strategy activated itself during operation without a restart, as intended by the dynamic field adaptation of Section 5.1. All six defects were corrected, and the results in this chapter refer to the corrected system. What this pattern means for the monitoring of an IdP is discussed in Section 7.1.

6.6 Threats to Validity

Several limits have already been stated together with the results they concern, for instance the shared clock of the correlation measurements or the resolution of the latency measurement. This section adds the limits that have not been stated there, most of which follow from the test environment and the generated traffic as a whole.

One host. All components ran on a single notebook, the IdP, the SPs and the log shipping stack in containers and the monitoring backend as a process on the host. This reproduces the logical separation of the backend from the IdP, but not the trust boundary that the design derives from it (Section 4.4). That an attacker who gains control of the IdP cannot alter the records that have already left it therefore follows from the design but was not tested.

Table 6.8: *Defects uncovered by the evaluation*

Defect	Consequence if undetected	Uncovered by
Inbound message identifier (`%I`) missing from the audit format	Exact correlation never active, all 3,690 correlations up to then made by the hashed heuristic	Correlation measurement
Heuristics compared against the Filebeat ingest time instead of the IdP event time	2,423 audit entries shared 448 time values, precision of the hashed heuristic 6 to 7 percent	Correlation measurement
Logstash set the field *encrypted* to false for every login	Every encrypted login reported as `UNENCRYPTED_ASSERTION`	End-to-end test against the clean SP
Version field missing from the stored behaviour profile	Profile updates discarded silently, so that `BEHAVIOUR_ANOMALY` compared against outdated profiles, while the unit test with a mocked repository passed	Integration test
Ingest path for the SP configuration inactive	Two rules evaluated an index that was no longer updated	Detection evaluation
Indexing errors in the analysis script of the performance measurement	Plausible but incorrect performance figures	Tracing values back to the raw measurement files

The shared host also affects the performance measurement. The backend and Elasticsearch processed the reports on the same processor as the IdP, and they had more to process in the configurations with interceptors than in the baseline. This competition was not measured, but where it has an effect, it makes the overhead appear larger rather than smaller. Absolute login durations are specific to this setup and are likely to be longer in production, where the network and the user store contribute. The differences between the configurations transfer better, since every run measured all three under the same conditions. On separate hosts, synchronous transmission would additionally wait for the network on both of its transmissions, so that its distance to asynchronous transmission would grow rather than shrink.

Scripted traffic. All logins were generated by scripts, either by the HTTP harness or by a browser driven by Selenium, with three test accounts, two SPs and a single authentication flow based on a password. Real users log in less regularly, through other flows such as a second factor, and from many different browsers and networks. Since no logins of real users were available, the evaluations show how the system behaves in the situations they reproduce, but not how often these situations arise in operation. This concerns how often logins at an SP follow each other closely enough to confuse the time-only correlation, which load a productive IdP reaches at its peak, and how often the rules detect real attacks or raise false alarms (Section 6.3.2). Single sign-on follow-up requests, which reuse an existing session, were not measured either. Since such requests are faster than a full login, the interceptors would account for a larger share of their duration.

Sequential load. In the performance series, each login started only after the previous one had completed, so that the IdP handled fewer than three logins per second and never two at once. The figures therefore describe the cost of a single login at low load. At a peak with many simultaneous logins, the additional processor time of the interceptors competes with other logins, and the estimate of about 1.4 additional logical processors at 50 logins per second assumes that the processor time per login does not grow with the load. Whether the backend accepts the reports quickly enough at such a peak was not tested either. Since the asynchronous interceptors give up a transmission after 1.5 s, a backend that falls behind would lose reports rather than slow down the logins.

Rules without an end-to-end test. Nine of the fifteen rules rest on the integration test, which passes its input directly to the rule engine and writes the stored history directly into Elasticsearch, and thus bypasses the log shipping stack. A fault in this stack, such as the one in Logstash described in Section 6.5, would remain undetected for these rules if it only affected fields that none of the six rules with an end-to-end test reads.

7 Discussion

This chapter interprets the results with respect to the research questions, discusses the structural limitations of the approach, and outlines directions for future work.

7.1 Interpretation of the Results

The results of this study can be read along the three research questions posed in Chapter 1. The first question asked which security relevant data can be collected at the application layer of a Shibboleth IdP and through which mechanisms. Three sources proved usable, namely the audit log, the configuration and metadata of the SPs, and the runtime state accessible through interceptors. The value of this combination does not primarily result from the number of sources but from the positions at which they observe the authentication process. The audit log records the outcome of a completed transaction, the configuration states what was intended for the partner involved, and the interceptors observe the incoming request and the outgoing response while the transaction is being processed. Each source therefore holds information that the others cannot provide, and the complementarity follows from their position in the processing sequence rather than from a deliberate division of responsibilities.

The second question asked how these data can be correlated in order to detect configuration weaknesses and anomalies. The correlation found a counterpart for every report, and with the inbound message identifier or at least one hash that separates the users, it found the correct one in every tested scenario. Only the strategy that relies on time alone (`TIME_WINDOW`) assigned reports to the wrong entry, and only where logins at the same SP followed each other within less than 100 ms. The heuristics remain reliable up to about three logins per second for a given combination of subject, address and SP, a rate that interactive users do not reach (Section 6.2.2). The fifteen detection rules behave as intended for the scenarios constructed for them. All of them are verified against a real Elasticsearch instance, six also from a real login, and legitimate logins raised no false alarms. This establishes the fidelity of the implementation, though not the detection quality in operation, a distinction developed in Section 6.3.2.

The third question asked what overhead the resulting system imposes. With asynchronous transmission, the latency of a login did not differ measurably from a deployment without interceptors. The point estimate is +6.2 ms, and the upper bound of the 95 % confidence interval is 17 ms, which is below five percent of the median login duration. The work inside the two interceptors amounts to about 0.45 ms. The cost appears as processor time instead, about 28 ms per login or 22 percent of the processor time of a login without interceptors, which the IdP spends in background threads. Synchronous transmission adds about 60 ms to every login and was the slower configuration in all 18 runs, without saving processor time. In the current implementation it would not improve the delivery either, since both modes only log a failed transmission (Section 6.4.2). The measurement therefore supports the choice of fire-and-forget transmission.

Beyond the individual research questions, one observation emerges across all three evaluations. Each of them uncovered at least one defect that operation itself had not revealed, six in total (Table 6.8). A missing token in the audit format had left the exact correlation permanently inactive, the log shipping stack would have reported every encrypted login as unencrypted, and indexing errors in the analysis script had produced plausible but incorrect performance figures. In none of these cases did the system fail visibly. It continued to run, alerts continued to be produced, and the dashboard continued to be populated. The defects became apparent only when the behaviour was compared against an expectation defined in advance.

This observation is worth stating separately because it bears on the premise of the work itself. The argument for observing an IdP from within rests on the assumption that a system which appears to function is not thereby known to function. These defects are instances of exactly that. They arose in the configuration, the pipeline and the analysis tooling of this work rather than in an attack scenario, which does not weaken the argument but illustrates its generality.

7.2 Limitations of the Approach

This section discusses the structural limitations of the proposed monitoring system. Rather than merely listing them, it analyses their underlying causes and shows how they follow from the position of the monitoring system within the IdP. It is therefore distinct from Section 6.6, which stated what the measurements of this work do not show. The limitations discussed here would persist irrespective of the effort invested, since they follow from where the observation takes place rather than from how thoroughly it was carried out.

As the assignment in Table 4.1 shows, the rules address two of the six SAML cross-protocol issues identified by Simpson and Groß [**FIMSurvey**], namely unencrypted communications exploited by message modification and lack of binding exploited by replay. The second is covered only in part, since a replayed assertion is presented to the SP and does not pass the IdP again, so that only the replay of a request remains observable (Table 6.5).

This limited coverage is not primarily caused by missing implementations but by the observation position of the monitoring system, which operates from within the IdP. The four remaining issues fall into three groups, namely those affecting a different system layer, those affecting a different party, and those observable only indirectly.

The first group consists of an issue that primarily affects a different system layer. DNS poisoning concerns name resolution and therefore a layer beneath the application [**FIMSurvey**]. The IdP processes the request it receives regardless of how the client resolved the corresponding address.

The second group consists of issues that affect a different party within the federation. Cross-Site Request Forgery (CSRF) describes an implementation vulnerability at the SP [**FIMSurvey**]. The relevant interaction occurs between the browser and the SP and therefore does not pass through the IdP. The same applies to message modification

enabled by a lack of binding, whose mitigation lies in the SP checking the credentials it receives [**FIMSurvey**].

The third group contains an issue that can be observed only indirectly from the IdP. A Man-in-the-Middle (MITM) attack is the issue in this category. It exploits a lack of binding, where a token is not tied to the SP for which it was issued, and the survey names binding the token to its intended SP as the countermeasure [**FIMSurvey**]. The attack itself takes place on the communication path after the message has left the IdP. The IdP can at most observe whether an outgoing assertion is restricted to its intended recipient, but not whether the SP enforces this restriction or whether a message was intercepted, and none of the implemented rules currently examines this condition.

The survey further lists attack classes that do not form cross-protocol issues and are therefore not part of the coverage figure. DDoS, XSS and session swapping occur at the network level, in the browser or at the SP and lie outside the observation point of the IdP for the same reasons [**FIMSurvey**]. Two further attack classes, UI redressing and ACS spoofing, were reported only for SAML and exploit a vulnerable IdP [**FIMSurvey**]. UI redressing loads a page of the IdP into an invisible frame and tricks the user into clicking it, and is mitigated by response headers such as `X-Frame-Options` [**GuardiansClouds**]. These headers are typically set by the web server in front of the IdP [**ShibbolethGIAC**] and lie outside the profile flows that the interceptors observe. ACS spoofing places an endpoint controlled by the attacker in the authentication request and succeeds if the IdP does not check it against the endpoints registered for the SP [**GuardiansClouds**]. Since the IdP sends responses only to endpoints enumerated in the SP's metadata [**MetadataForSP**], a rule comparing the requested endpoint with the metadata would be conceivable, but it is not part of the current catalogue.

Two cases are considered separately because of their particular relevance. The first is the bogus merchant scenario, which is the only cross-protocol issue in the survey that concerns Shibboleth directly and is nevertheless not covered by the monitoring system. The reason is that the malicious SP is a regular federation participant, registered in the federation metadata as a legitimate partner. From the perspective of the IdP it therefore need not exhibit any observable property that distinguishes it from other legitimate participants. Monitoring changes to the configuration and metadata of an SP could provide an additional source of information, since in the absence of a basis for prior trust a change is the only usable signal. Such a mechanism would, however, detect the transition of a provider rather than a provider that was malicious from the outset.

The second case is more fundamental. The malicious provider scenario describes an IdP that abuses its own position within the federation. Since the monitoring system is integrated into the IdP, the observer and the potential attacker reside within the same system. An entity that fully controls the IdP therefore also controls the collection layer of the monitoring system, including which records are produced and which are not. This limitation follows directly from the observation point and cannot be overcome by adding further detection rules within the same trust domain. It is also not specific to this work. Snodgrass et al. make the same assumption in their threat model, where only services outside the compromised system remain trustworthy once an adversary has gained control

of it [**TamperDetection**]. Detecting this class of behaviour would require an observation point outside the IdP, for instance at the SPs or at the level of the federation itself.

A final limitation connects back to the motivation of this work. The risk classification used for the severity assignment in Section 4.5.3 distinguishes five levels, of which the highest describes an adversary able to authenticate as any user at any SP. None of the fifteen rules reaches this level, since it presupposes a compromise of the signing key. An adversary in possession of that key produces formally valid assertions that are indistinguishable from legitimate ones by any of the implemented rules. The Sunburst incident described in Chapter 1 is precisely such a case and would therefore not have been detected by this system. What the system can detect are configuration weaknesses that make an attack of this kind more likely to succeed or harder to notice, which is a narrower claim than the motivation might initially suggest.

7.3 Implications for Future Research

The findings and limitations identified in this study point to several directions for future work in the area of application layer monitoring for federated identity infrastructures. They fall into two categories, namely extensions that would widen the observation point and questions that cannot be answered without a different kind of data.

A primary direction is to extend the detection capabilities beyond the IdP and incorporate observations made at the SP. The approach presented here is inherently limited by its position, as discussed in Section 7.2, and several of the uncovered attack classes are located precisely where the IdP has no visibility. Martín et al. have proposed behavioural anomaly detection on the relying party side [**UEBAFederation**], arguing that delegating authentication does not transfer the entire security responsibility. Combining both perspectives would address opposite ends of the same federation. Such an extension would require a suitable and preferably standardised integration mechanism for SPs, which introduces organisational challenges beyond the technical ones, since it presupposes cooperation between parties that are administratively independent.

A second direction concerns the observation point within the IdP itself. The outbound interceptor added in this work makes the response level fields available from a second, independent source, and it enables the comparison with the audit entry described in Section 4.5.2. What it cannot provide are the assertion internal fields of encrypted assertions, since the encryption has already taken place at that hook. An observation point before the encryption would close this gap, but the interception points that Shibboleth offers to developers do not include one, and an intervention in the profile flow would conflict with the third requirement from Section 4.1. Whether the audit extraction points of the IdP could serve this purpose is a question for future work.

A third direction addresses the one Shibboleth specific issue that remains uncovered. The system already receives every changed configuration and metadata document of an SP (Section 5.1), but no rule reports the change itself. Such a rule would provide a partial answer to the bogus merchant scenario, since a change is the only observable signal where no basis for prior trust exists. What such a mechanism could establish and what it could

not is discussed in Section 7.2, and the distinction would need to be stated carefully in any implementation.

A fourth direction is the extension of the approach to OpenID Connect (OIDC) [**OpenIDConnectCore**]. The present work focuses exclusively on SAML and Shibboleth, so that it remains an open question whether the three source architecture developed here transfers to OIDC based infrastructures. The architecture itself appears protocol independent, since an OIDC provider likewise maintains a registration of its clients, writes records of completed transactions and processes incoming requests at runtime. The detection rules, by contrast, are protocol specific and would not transfer, since conditions such as a missing proof key exchange or an implicit flow have no direct counterpart in the rules developed here [**RFC7636**, **RFC9700**]. Future work could therefore examine which application level sources exist in an OIDC provider, whether they are complementary in the same sense, and which rules follow from the documented weaknesses of that protocol.

Finally, one question cannot be addressed by extending the system at all. Determining how reliably the rules detect genuine attacks requires productive traffic in which each authentication is independently labelled, as discussed in Section 6.3.2. Producing such a dataset is a research problem in its own right rather than an implementation task, and it would benefit not only this work but any evaluation of detection systems in federated identity infrastructures, where the same constraint applies.

8 Conclusion

This thesis presented an application layer monitoring tool for the Shibboleth IdP that detects documented vulnerabilities, configuration weaknesses and anomalous authentication behaviour by combining three separate sources of information from within the IdP. The collected data is transmitted to an external backend, where it is stored, correlated and evaluated against a catalogue of fifteen detection rules.

The three sources proved complementary, and the reason is more specific than their number. The audit log records the outcome of a completed transaction, the configuration states what was intended for the partner involved, and the interceptors observe the incoming request and the outgoing response while the transaction is being processed. What each source can see follows from its position in the processing sequence, which is why their combination permits comparisons that no single source supports on its own. The comparison between a declared configuration and an observed event is the clearest example, since a login without multi factor authentication is unremarkable until it is related to a service provider that requires it.

The detection rules are derived rather than assembled from experience. Thirteen of the fifteen rules can be traced to vulnerabilities and attack classes documented in the literature, to security recommendations for SAML deployments or, for the two rules that compare independently produced records, to the log integrity argument. The two rules that cannot be traced are named as such. All fifteen are verified against a real Elasticsearch instance, and six additionally through the complete pipeline, four of them from a browser login up to the resulting alert.

The correlation links the records of a login reliably. All 7,236 reports of the performance series found their counterpart, and with the identifier of the request, or with the subject and the address as long as one of them separates the users, the correct one was found in every tested scenario. Only the strategy based on time alone (`TIME_WINDOW`) confused logins, and only those that followed each other at the same SP within less than 100 ms, so that operators should record the identifier in the audit format, or at least the subject and the address.

In the asynchronous transmission mode, the two interceptors introduced no measurable latency overhead. The difference to a deployment without interceptors is +6.2 ms with an upper bound of 17 ms for the 95% confidence interval, and the work performed inside the interceptors amounts to about 0.13% of the login duration. Their cost appears as processor time instead, about 28 ms per login, while blocking transmission would add about 60 ms to each login without saving any of it. The approach can therefore be integrated into the authentication workflow of a central authentication service without a perceptible effect on the logins it observes, provided that the records are transmitted asynchronously.

The comparison of two independently produced records of the same event serves as a check of log integrity without changing the logging chain. For 200 legitimate logins the consistency check between outbound report and audit entry raised no alert, and it reported exactly the modified field of a deliberately manipulated audit entry. The check

shows that two records disagree, but not which of them was altered, and it cannot see the fields inside an encrypted assertion.

A recurring observation across the evaluation deserves mention, since it bears on the premise of the work. In each of the three evaluations, the measurement uncovered a defect that operation itself had not revealed. The system continued to run in every case, alerts continued to be produced and the dashboard continued to be populated. The defects became apparent only when the behaviour was compared against an expectation defined in advance. That a system appears to function is therefore not evidence that it does, which is the same argument that motivates observing an IdP from within in the first place.

The study also identified limitations that follow from the observation point rather than from the effort invested. Several federation level attack scenarios occur at a different system layer, involve a different party, or are observable only through their preconditions. One case is more fundamental, since a malicious IdP cannot be detected by a system running inside it, where observer and adversary coincide. The severity classification makes a further boundary explicit, as none of the fifteen rules reaches the highest risk level, which presupposes a compromise of the signing key. The Sunburst incident that motivated this work would consequently not have been detected by the system presented here, although the configuration weaknesses that make such an attack easier to carry out and harder to notice are within its reach.

What this thesis contributes is therefore not a new architecture for collecting and correlating security events, since that is established practice. It is the demonstration that an IdP holds security relevant information which no log consuming system can obtain, an analysis of which properties are observable from which position, and an implementation that makes this information available at a cost that can be stated in numbers rather than assumed.

List of Figures

2.1 SP-Initiated SSO with Redirect and POST Bindings [**SAMLOverview**] . . . 7

4.1 UML Deployment Diagram of the Proposed Monitoring System 18

List of Tables

4.1 Assignment of the 15 Detection Rules to Documented Vulnerabilities and Attack Classes 27
4.2 Severity assigned by each rule 29

6.1 Datasets of the correlation measurements 46
6.2 Precision of the heuristic strategies per dataset 46
6.3 Precision of `TIME_WINDOW` by distance to the nearest other login at the same SP 47
6.4 Offset between the reception of a report and the event time of its audit entry 48
6.5 End-to-end coverage per rule 51
6.6 Consistency check for legitimate logins 52
6.7 Cost of a login per configuration 55
6.8 Defects uncovered by the evaluation 57

Glossary

ACS Assertion Consumer Service.

CSRF Cross-Site Request Forgery.

EDR Endpoint Detection and Response.

FIM Federated Identity Management.

HMAC Hash-based Message Authentication Code.

ID-FF Identity Federation Framework.
IdP Identity Provider.
IDS Intrusion Detection System.

LDAP Lightweight Directory Access Protocol.

MITM Man-in-the-Middle.

OASIS Organization for the Advancement of Structured Information Standards.
OIDC OpenID Connect.

SAML Security Assertion Markup Language.
SIEM Security Information and Event Management.
SP Service Provider.
SSO Single Sign On.

WAF Web Application Firewall.

XSW XML Signature Wrapping.